“Examination Management System”

A

Project Report

Submitted in partial fulfillment of the requirement for the award of

Degree of

Bachelor of Science in Information Technology

Submitted to

444 - SWATANTRATA-SENANI SHANKARLAL AGRAWAL PRABANDHAN SAH TAKNIKI MAHAVIDYALAY, PATNA

Submitted by

Shrawan Kumar Jha

Registration No – 202344400191

B.Sc.IT, 3rd Year

Session – 2023 – 2026

Under Joint Guidance of

Prof. Anjesh Kumar

& Prof. Niraj Kumar Singh

[Asst. Professor]

Computer Science &

Applications

444 - SWATANTRATA-SENANI SHANKARLAL AGRAWAL PRABANDHAN SAH TAKNIKI MAHAVIDYALAY, PATNA

Session 2023-26

Date:……

CERTIFICATE

This is to certify that the work embodies in this project entitled, **"EXAMINATION MANAGEMENT SYSTEM ",** being submitted by **Shrawan Kumar Jha** (Registration No: 202344400191) in partial fulfillment of the requirement for the award of **"Bachelor of Science in Information Technology"** to Patliputra University, Patna during the academic year **2023-26** is a record of bonafide piece of work, carried out by him/her under our/my supervision and guidance in the

"Department of Computer Science & information Technology", 444 - SWATANTRATA-SENANI SHANKARLAL AGRAWAL PRABANDHAN SAH TAKNIKI MAHAVIDYALAY, PATNA

Dr. Prof. Neeraj Agrawal	**Dr. Prof. Neeraj Kumar**	**Dr. Prof. Amit Kumar Shukla**
Director	**Dean - Academics**	**Head of Department-IT**

444 - SWATANTRATA-SENANI SHANKARLAL AGRAWAL PRABANDHAN SAH TAKNIKI MAHAVIDYALAY, PATNA

Session 2023-26

Date:----------

APPROVAL CERTIFICATE

The project report entitled **"EXAMINATION MANAGEMENT SYSTEM",** being submitted by **Shrawan Kumar Jha** (Registration Number -- 202344400191) has been examined by us and is hereby approved for the award of degree "Bachelor of Science in Information Technology", for which it has been submitted. It is understood that by this approval the undersigned do not necessarily endorse or approve any statement made, the opinion expressed or conclusion drawn therein, but approve the project only for the purpose for which it has been submitted.

(Internal Examiner) **(External Examiner)**

Date: **Date:**

444 - SWATANTRATA-SENANI SHANKARLAL AGRAWAL PRABANDHAN SAH TAKNIKI MAHAVIDYALAY, PATNA

Session 2023-26

Date:----------

DECLARATION

I, **SHRAWAN KUMAR JHA** hereby declare that the work, which is being presented in the project report entitled "EXAMINATION MANAGEMENT SYSTEM", partial fulfillment of the requirements for the award of degree of "**Bachelor of Science in Information Technology**" submitted in the department of **Computer Science & Information Technology (444 - SWATANTRATA-SENANI SHANKARLAL AGRAWAL PRABANDHAN SAH TAKNIKI MAHAVIDYALAY, PATNA)** is an authentic record of my own work carried under the guidance of **Dr. prof. Amit Shukla Sir**. I have not submitted the matter embodied in this report for the award of any other degree.

Submitted by

Shrawan Kumar Jha

Registration No – 202344400191

B.Sc.IT, 3rd Year

Session – 2023 – 2026

ACKNOWLEDGEMENT

"A journey is easier when you travel together. Interdependence is certainly more valuable than independence."

I would like to thanks, **Prof. Niraj Kumar Singh Sir** and **Prof. Ravi Soni Sir**, for providing regular guidance and insight into my project work. I also thank them for all advice he has given me in the past year, and for always having time for me, whenever I needed. I give special thanks to **Dr. Prof. Amit Kumar Shukla Sir, Prof. Anjesh Sir, Prof. Pankaj Chandra Sinha** and **Prof. Raju Upadhyay Sir** for always being willing to help find solutions to any problems I had with my work. "The completion of any project depends upon the cooperation, coordination, and combined efforts of several resources of knowledge, inspiration, and energy".

I also extend my deepest gratitude to **Director, CIMAGE GROUP OF INSTITUITIONS, PATNA, Dr. Prof. Neeraj Agrawal Sir**, **Dean CIMAGE GROUP OF INSTITUITIONS, PATNA, Dr. Prof. Neeraj Kumar Sir** and **Centre Head Mrs. Megha Agrawal Ma'am** for providing all the necessary facilities and true encouraging environment to bring out the best in my endeavors.

I express my gratitude and thanks to all the staff members of Computer Science departments for their sincere cooperation in furnishing relevant information to complete this Project well in time successfully.

I extend a special word to my friends, who have been a constant source of inspiration throughout my project work. Lastly but not least I must express my cordial thank to my parent and family members who gave me the moral support without that it is impossible to complete my project work. With this note, I thank everyone for the support.

Submitted by

Shrawan Kumar Jha

Registration No – 202344400191

B.Sc.IT, 3rd Year

Session – 2023 – 2026

INDEX TABLE: PAGE NO

Introduction…… 1

Objective…… 1 – 2

Project Overview…….... 2

Feasibility Study…… 2 – 4

System Analysis……... 4 – 5

System Specifications……..…… 5

Description Of Technology Used…… 5 – 19

Design…… 19 – 20

Project Planning & Scheduling…… 20

Gantt Chart…… 21

Pert Chart…….. 22 – 23

DFD…… 23 – 26

Module Description…… 27 – 28

Data Dictionary…… 28 – 30

ER Diagram……...30 – 32

Scope and Future Enhancement……... 33

Implementation Methodology…… 33 – 34

Testing……... 34 – 36

Limitations and Further Enhancements……... 36 – 37

Security Mechanism…… 37 – 38

Database Model…….. 38 – 39

Files of the project in Image……..…… 40

Teacher Module Code & Snapshots…… 41 – 86

Student Module Code & Snapshots…….. 86 - 147

Conclusion…… 147

Bibliography…… 148

INTRODUCTION:

The rapid growth of digital technology and the widespread use of the Internet have significantly transformed traditional systems into online platforms. One such transformation can be observed in the field of education, where online examination systems are becoming increasingly popular and essential. Educational institutions and organizations are now adopting web-based solutions to conduct examinations efficiently, securely, and remotely.

The objective of this project is to develop a comprehensive **Examination Management System** that allows students to take exams from anywhere using the Internet. This system provides a virtual platform where students can log in, access available examinations, and attempt them within a specified time limit. The system is designed to ensure fairness, security, and ease of use for both students and teachers.

In this system, teachers can create and manage examinations by adding questions, setting time limits, and publishing exams for students. Questions can be of different types such as multiple-choice questions (MCQs) and direct answer questions. Students can give available exams, attempt them, and submit their responses online.

During the examination process, the system records student responses, tracks time, and ensures that the exam is completed within the given duration. Additional features such as attempt tracking, performance evaluation, and feedback submission enhance the overall functionality of the system.

After submission, the system evaluates the responses and generates results or stores the data for further analysis. Teachers can view student performance, analyze statistics, and manage students effectively. The system also includes authentication mechanisms to ensure that only authorized users can access specific functionalities.

Overall, the Examination Management System provides a reliable, efficient, and secure solution for conducting examinations digitally, reducing manual effort, saving time, and improving accessibility for both students and educators.

OBJECTIVE:

The Online Examination Management System is mainly useful for students and educational institutions who want to conduct and attend examinations without the limitations of time and location. This system allows students to appear for exams from anywhere using the Internet, without the need to be physically present in a classroom.

Even during any time of the day, whether morning or night, students can log in to the system and attempt their assigned examinations within the given schedule. The platform provides various types of questions such as multiple-choice questions (MCQs) and direct answer questions, making the examination process flexible and effective.

The system is designed with the principle that **"User convenience and accessibility come first."** It ensures that students can easily navigate, attempt exams, and submit their responses

without difficulty. At the same time, teachers can efficiently create exams, manage questions, and monitor student performance.

The system also includes secure login, time tracking, and result evaluation features to maintain the integrity of the examination process. It reduces manual work, saves time, and eliminates the need for paper-based exams.

Overall, the objective of this system is to provide a reliable, user-friendly, and secure platform for conducting online examinations, ensuring a smooth experience for both students and educators.

PROJECT OVERVIEW:

Once a user enters the system with their own username and password, the system authenticates the user and provides access based on their role (student or teacher). After successful login, the user is directed to their respective dashboard where all functionalities are available.

In case of a student, once they log in, they can view previous exams max result and give feedback, When a student starts an exam, the system records the attempt and begins tracking time automatically. If the student answers questions, those responses are stored securely in the system.

If a student wishes to change an answer, there is no provision to move back to previously attempted questions. Once the exam is submitted, the system evaluates the responses or stores them, and the result is displayed instantly.

In case a student logs out or the session is interrupted before submitting the exam, the student must restart the exam from question number 1.

For teachers, after logging in, they can create exams, add questions, manage students, and view detailed statistics of student performance and also they print the exam report into pdf. They can also analyze attempts.

The system ensures smooth interaction, provides an OTP-based password changing option for both teachers and students, ensures secure data handling, and enables efficient management of the entire examination process, thereby offering a complete digital solution for conducting exams.

FEASIBILITY STUDY:

Feasibility study is an important step in software development that determines whether a proposed system is practical and beneficial to implement. It evaluates different aspects of a project to ensure that it can be successfully developed within the available resources and constraints.

In the case of the **Examination Management System**, the feasibility study helps in analyzing whether the system can be developed efficiently and whether it will meet the requirements of students, teachers, and administrators.

The feasibility of the system is analyzed under the following dimensions:

1. Technical Feasibility

Technical feasibility checks whether the required technology and resources are available to develop the system.
The proposed system is developed using technologies such as **HTML, CSS, JavaScript, Node.js, Express.Js, and MongoDB**, which are widely used and easily available. The system can run on standard computers and does not require any specialized hardware.
Hence, the project is technically feasible.

2. Operational Feasibility

Operational feasibility determines whether the system will be accepted and used by the users.
The system provides a user-friendly interface for students and teachers, allowing them to easily conduct and attempt exams. Features like login, exam interface, and result display are simple and easy to use.
Therefore, the system is operationally feasible.

3. Economic Feasibility

Economic feasibility evaluates the cost required to develop and maintain the system.
Since the project uses open-source technologies and does not require expensive tools or hardware, the development cost is minimal. Maintenance cost is also low.
Thus, the system is economically feasible.

4. Social Feasibility

Social feasibility checks whether the system is acceptable to society.
The system promotes digital learning and reduces the need for paper-based examinations, making it environmentally friendly. It also supports remote examinations, which is beneficial for students.
Hence, the system is socially feasible.

5. Management Feasibility

Management feasibility ensures that the project can be managed effectively.
The project follows proper planning, scheduling, and development phases, making it easy to manage and monitor. The system aligns with educational needs and institutional requirements.
Thus, the system is management feasible.

6. Legal Feasibility

Legal feasibility checks whether the system complies with legal and ethical standards.
The system ensures secure handling of user data and follows basic data privacy principles.
No illegal content or activities are involved.
Therefore, the system is legally feasible.

7. Time Feasibility

Time feasibility determines whether the project can be completed within the given time frame.
The project is planned and scheduled properly using tools like Gantt Chart and PERT Chart, ensuring timely completion within three months.
Hence, the system is time feasible.

SYSTEM ANALYSIS:

System Analysis is the process of studying and analyzing an existing system with the aim of improving it or developing a new system. It involves understanding user requirements, identifying problems in the current system, and proposing a suitable solution. It acts as a bridge between system requirements and system design.

1. Existing System

The existing system for conducting examinations is mostly manual or semi-digital. In traditional methods, examinations are conducted using paper-based formats where teachers prepare question papers, students write answers manually, and evaluation is done later by teachers.

Managing records such as student performance, answer sheets, and results becomes difficult in this system. Maintaining historical data requires physical storage, which consumes space and is prone to damage or loss.

2. Disadvantages of Existing System

The following are the major disadvantages of the existing system:

- It is difficult to maintain and manage large amounts of student and examination data manually.
- More time and manual effort are required to conduct exams and evaluate answer sheets.
- Managing historical records such as past results and answer sheets is tedious and requires significant physical storage.
- There is a higher chance of human errors during evaluation and data handling.
- Generating reports and analyzing student performance is time-consuming.
- Lack of security and transparency in the examination process.

3. Objective of the Proposed System

The objective of the Online Examination Management System is to provide a digital platform for conducting examinations efficiently and securely.

The system aims to simplify the process of exam creation, management, and evaluation. It allows teachers to create exams, add questions, and monitor student performance, while students can attempt exams online from any location.

The system ensures accurate result generation, secure data handling, and easy access to historical records. It reduces manual work, saves time, and improves overall efficiency in managing examinations.

SYSTEM SPECIFICATIONS:

Hardware Requirements

- Processor: Dual Core or higher
- RAM: 4 GB or higher
- Hard Disk: 20 GB or more
- Keyboard: Standard Keyboard
- Internet Connection

Software Requirements

- **Backend Technology:** Node.js with Express.js
- **Frontend Technology:** HTML, CSS, JavaScript (Vanilla JS)
- **Database:** MongoDB
- **Authentication:** JWT (JSON Web Token)
- **Email Service:** Nodemailer
- **PDF Generation:** jsPDF
- **Screen Capture / Report Export:** html2canvas
- **Server Environment:** Node.js Runtime
- **Development Tools:** VS Code, Postman
- **Web Browser:** Google Chrome, Mozilla Firefox, Microsoft Edge

DESCRIPTION OF TECHNOLOGIES USED:

Backend Technology: Node.js with Express.js

Introduction

Node.js is a JavaScript runtime environment that allows developers to run JavaScript on the server side. It is built on Chrome's V8 engine and is widely used for building fast and scalable web applications.

Express.js is a lightweight web framework built on top of Node.js that simplifies backend development by providing powerful features like routing, middleware, and API handling.

In the Examination Management System, Node.js with Express.js is used to handle all server-side operations such as user authentication, exam management, data processing, and communication with the database.

Advantages of Node.js with Express.js

Fast Execution Speed:
Node.js uses a non-blocking, event-driven architecture which makes it very fast. It can handle multiple requests simultaneously without slowing down the system.

Single Language (JavaScript):
Both frontend and backend use JavaScript, which makes development easier and reduces complexity.

Scalability:
Node.js is highly scalable and suitable for applications with a large number of users, such as online examination systems.

Lightweight Framework (Express.js):
Express.js is simple and minimal, making it easy to build APIs and manage routes efficiently.

Efficient API Handling:
Express.js provides easy methods to handle HTTP requests (GET, POST, PUT, DELETE), which helps in building RESTful APIs.

Middleware Support:
It allows the use of middleware functions for tasks like authentication, logging, and error handling.

Real-Time Processing:
Node.js supports real-time data processing, which is useful for features like live exams and instant result generation.

Large Community Support:
Node.js has a huge community, which provides libraries, tools, and support for faster development.

Easy Integration with Database:
Node.js can easily connect with MongoDB and perform CRUD operations efficiently.

High Performance:

Due to its asynchronous nature, it provides better performance compared to traditional server technologies.

Frontend Technology: HTML, CSS, Javascript (Vanilla Js)

Introduction

HTML (HyperText Markup Language), CSS (Cascading Style Sheets), and JavaScript are the core technologies used for developing the frontend of web applications. These technologies are responsible for creating the structure, design, and interactivity of the system.

HTML is used to define the basic structure of web pages, CSS is used to design and style the user interface, and JavaScript is used to add dynamic behavior and interactivity.

In the Examination Management System, these technologies are used to create a responsive and user-friendly interface for students and teachers to interact with the system, such as login, exam attempt, and result viewing.

Advantages of HTML, CSS, and JavaScript

Simple and Easy to Use:

HTML, CSS, and JavaScript are easy to learn and widely used, making development faster and more efficient.

Platform Independent:

These technologies work on all modern web browsers and operating systems without any compatibility issues.

Responsive Design:

CSS allows the creation of responsive layouts that adjust to different screen sizes such as mobile, tablet, and desktop.

Dynamic and Interactive:

JavaScript enables dynamic features such as form validation, real-time updates, and user interaction without reloading the page.

Lightweight and Fast:

Vanilla JavaScript does not require any external libraries, making the application lightweight and fast.

Separation of Design and Structure:

HTML handles structure while CSS manages design, which improves code readability and maintainability.

Full Control Over Code:

Using Vanilla JavaScript provides complete control over the application without dependency on external frameworks.

Efficient DOM Manipulation:

JavaScript allows modification of webpage content dynamically, improving user experience.

Cross-Browser Support:

Frontend technologies are supported by all major browsers like Chrome, Firefox, and Edge.

Improved User Experience:

These technologies together provide a smooth, interactive, and visually appealing interface.

Database: Mongodb

Introduction

MongoDB is a NoSQL database that stores data in a flexible, JSON-like format called documents. Unlike traditional relational databases, MongoDB does not use tables and rows; instead, it uses collections and documents, which makes data storage more flexible and scalable.

It is designed to handle large amounts of data efficiently and is widely used in modern web applications.

In the Examination Management System, MongoDB is used to store user information, exam details, questions, student attempts, results, feedback, and password reset data.

Advantages of MongoDB

Flexible Schema:

MongoDB allows dynamic schema design, which means data structure can be modified easily without affecting existing data.

High Scalability:

It supports horizontal scaling, making it suitable for applications with a large number of users and data.

Fast Data Access:

MongoDB provides high performance for read and write operations, which is important for real-time applications.

JSON-like Data Storage:

Data is stored in BSON (Binary JSON) format, which is easy to understand and works well with JavaScript-based applications.

Easy Integration with Node.js:

MongoDB integrates smoothly with Node.js, allowing efficient data handling and faster development.

No Complex Joins:

MongoDB reduces the need for complex joins by using embedded documents and references.

High Availability:

It provides features like replication and automatic failover to ensure data availability.

Efficient Querying:

MongoDB supports powerful query operations for filtering, sorting, and aggregating data.

Better Performance for Large Data:

It performs well with large datasets and high traffic applications.

Open Source:

MongoDB is free and open-source, making it accessible for developers.

Authentication: Jwt (Json Web Token)

Introduction

JWT (JSON Web Token) is a secure method used for authentication and authorization in web applications. It is a compact, URL-safe token that is used to verify the identity of users.

When a user logs in successfully, the server generates a token and sends it to the client. This token is then stored on the client side and used in subsequent requests to access protected resources.

In the Examination Management System, JWT is used to authenticate users (students and teachers) and ensure that only authorized users can access specific features of the system.

Advantages of JWT

Stateless Authentication:

JWT does not require storing session data on the server, making the system more scalable and efficient.

Secure Data Transmission:

JWT uses encryption and digital signatures to ensure that the data is secure and cannot be tampered with.

Compact and Lightweight:

JWT tokens are small in size and can be easily transmitted in HTTP headers, making them efficient for web applications.

Fast Authentication:

Since no database lookup is required for every request, authentication becomes faster.

Cross-Platform Support:

JWT can be used across different platforms such as web, mobile, and APIs.

Easy Implementation:

JWT is simple to implement and widely supported in many programming languages.

Role-Based Access Control:

JWT can store user roles (student/teacher), allowing access control for different functionalities.

Reduced Server Load:

As it is stateless, it reduces the load on the server by avoiding session storage.

Scalability:

JWT supports distributed systems and microservices architecture easily.

Reliable and Standardized:

JWT is an industry-standard method for secure authentication.

Email Service: Nodemailer

Introduction

Nodemailer is a Node.js module used for sending emails from web applications. It allows developers to send emails easily using SMTP or other transport services.

It is widely used for features like email verification, notifications, and password recovery.

In the Examination Management System, Nodemailer is used to send OTP emails for password reset and user verification, ensuring secure communication between the system and users.

Advantages of Nodemailer

Easy to Use:

Nodemailer provides a simple and straightforward way to send emails using Node.js.

Supports Multiple Email Services:

It can work with various email providers such as Gmail, Outlook, and custom SMTP servers.

Secure Email Sending:

Supports secure protocols like SSL/TLS, ensuring safe transmission of emails.

OTP and Verification Support:

Useful for sending OTPs for password reset and account verification.

Customizable Emails:

Allows sending formatted emails using HTML and attachments.

Lightweight and Efficient:

Nodemailer is fast and does not add heavy load to the application.

Integration with Node.js:

Easily integrates with backend systems built using Node.js and Express.js.

Supports Attachments:

Allows sending files and documents along with emails.

Reliable Delivery:

Ensures proper delivery of emails with error handling support.

Open Source:

Freely available and widely used by developers.

Pdf Generation: Jspdf

Introduction

JsPDF is a JavaScript library used to generate PDF files directly in the browser. It allows developers to create, format, and download PDF documents dynamically without requiring any server-side processing.

It is widely used for generating reports, invoices, and downloadable documents in web applications.

In the Examination Management System, jsPDF is used to generate result reports and exam summaries that can be downloaded and printed by users.

Advantages of jsPDF

Client-Side PDF Generation:

JsPDF generates PDFs directly in the browser without needing a server, making the process faster and efficient.

Easy to Use:

It provides simple functions to create and customize PDF documents.

Lightweight Library:

JsPDF is small in size and does not affect application performance significantly.

Customizable Content:

Allows adding text, images, tables, and formatting to PDFs.

Download and Print Support:

Users can easily download and print generated PDF files.

Integration with JavaScript:

Works seamlessly with JavaScript and frontend frameworks.

Fast Processing:

Generates PDF files quickly without delay.

No Server Dependency:

Reduces server load by handling PDF generation on the client side.

Flexible Design:

Developers can design PDFs according to application requirements.

Open Source:

JsPDF is free and widely supported by the developer community.

Screen Capture / Report Export: Html2canvas

Introduction

html2canvas is a JavaScript library that allows capturing screenshots of web pages or specific elements directly in the browser. It converts HTML content into a canvas image, which can then be saved or used for further processing.

It is commonly used for generating visual reports, exporting UI content, and capturing webpage data as images.

In the Examination Management System, html2canvas is used to capture result pages and generate image-based reports, which can be saved or downloaded by users.

Advantages of html2canvas

Client-Side Screen Capture:

Captures screenshots directly in the browser without requiring any server-side processing.

Easy to Use:

Provides simple functions to capture HTML elements and convert them into images.

No External Tools Required:

Does not require additional software or plugins for screen capturing.

Supports Full Page and Partial Capture:

Can capture the entire webpage or specific sections such as result cards.

Integration with jsPDF:

Works well with jsPDF to generate PDFs from captured images.

Lightweight and Efficient:

Does not significantly impact application performance.

Improves User Experience:

Allows users to download or save reports easily.

Cross-Browser Support:

Works on most modern web browsers.

Customizable Output:

Developers can control image quality, format, and layout.

Open Source:

Freely available and widely used in web applications.

Server Environment: Node.Js Runtime

Introduction

Node.js runtime is the environment in which server-side JavaScript code is executed. It is built on Chrome's V8 JavaScript engine and allows developers to run JavaScript outside the browser.

It provides an event-driven and non-blocking architecture, which makes it highly efficient for handling multiple requests simultaneously.

In the Examination Management System, the Node.js runtime is used to execute backend code, manage server operations, handle API requests, and communicate with the MongoDB database.

Advantages of Node.js Runtime

High Performance:

Node.js uses the V8 engine, which compiles JavaScript into machine code, resulting in fast execution.

Non-Blocking Architecture:

It uses asynchronous processing, allowing multiple requests to be handled at the same time without waiting.

Event-Driven Model:

Efficiently handles concurrent connections using an event-driven approach.

Scalability:

Suitable for applications with high traffic and multiple users, such as online examination systems.

Lightweight and Efficient:

Consumes fewer system resources compared to traditional server environments.

Cross-Platform Support:

Runs on Windows, Linux, and macOS, making it flexible for developers.

Real-Time Capability:

Supports real-time features like live updates and instant responses.

Easy Integration:

Works smoothly with databases like MongoDB and frameworks like Express.js.

Large Ecosystem (NPM):

Provides access to a vast number of libraries and packages for faster development.

Open Source:

Free to use and supported by a large developer community.

Development Tools: Vs Code, Postman

Introduction

Development tools are essential software applications that help developers write, test, and manage code efficiently. In this project, tools like Visual Studio Code (VS Code) and Postman are used to improve productivity and ensure proper development and testing of the system.

VS Code is a powerful code editor used for writing and managing frontend and backend code, while Postman is used for testing APIs and verifying server responses.

Advantages of VS Code

User-Friendly Interface:

VS Code provides a simple and clean interface, making it easy for developers to write and manage code.

Extension Support:

It supports a wide range of extensions for different programming languages and frameworks.

Built-in Debugging:

Allows developers to debug code efficiently and fix errors quickly.

IntelliSense (Auto Suggestion):

Provides smart code suggestions, improving coding speed and accuracy.

Version Control Support:

Integrated Git support helps in managing code versions and collaboration.

Lightweight and Fast:

Consumes less memory and runs smoothly on most systems.

Multi-language Support:

Supports multiple programming languages like JavaScript, HTML, CSS, etc.

Advantages of Postman

API Testing:

Postman allows testing of APIs by sending HTTP requests such as GET, POST, PUT, and DELETE.

User-Friendly Interface:

Provides an easy interface to test and analyze API responses.

Fast Debugging:

Helps in quickly identifying issues in backend APIs.

Automation Support:

Supports automated testing of APIs.

Environment Management:

Allows managing different environments like development and production.

Response Analysis:

Displays response data clearly, including status codes and JSON output.

Collaboration:

Supports sharing APIs and collections with team members.

Web Browsers: Google Chrome, Mozilla Firefox, Microsoft Edge

Introduction

Web browsers are software applications used to access and display web pages on the internet. They play an important role in running and testing web applications.

In the Examination Management System, web browsers are used to access the application, interact with the user interface, and test the system across different environments. Popular browsers such as Google Chrome, Mozilla Firefox, and Microsoft Edge ensure compatibility and smooth performance.

Advantages of Web Browsers

User-Friendly Interface:

Modern web browsers provide a simple and easy-to-use interface for accessing web applications.

Cross-Browser Compatibility:

Using multiple browsers helps ensure that the application works correctly on different platforms.

Fast Performance:

Browsers like Chrome and Edge use advanced engines that provide fast loading and smooth execution of web pages.

Developer Tools:

Built-in developer tools help in debugging, inspecting elements, and testing the application.

Security Features:

Browsers provide security features such as safe browsing, HTTPS support, and protection against malicious websites.

Responsive Testing:

Allow testing of web applications on different screen sizes and devices.

Support for Modern Web Technologies:

Browsers support the latest HTML, CSS, and JavaScript features.

Extension Support:

Provide extensions and plugins that enhance development and testing.

Cross-Platform Availability:

Available on multiple operating systems such as Windows, Linux, and macOS.

Regular Updates:

Browsers are frequently updated to improve performance and security.

DESIGN:

INTRODUCTION

Design is the first step in the development phase of any system. It involves defining the structure, components, and workflow of the system in sufficient detail so that it can be successfully implemented. In the context of the Online Examination Management System, design plays a crucial role in transforming user requirements into a functional and efficient software solution.

Once the system requirements are analyzed and clearly defined, the design phase begins. It mainly includes three important activities: system design, coding, and testing, which together help in building and verifying the system. The design phase ensures that all functionalities such as user authentication, exam creation, question management, and result processing are properly structured.

The design activities are of great importance because the decisions made during this phase directly affect the performance, reliability, and maintainability of the system. A well-designed system ensures smooth interaction between different modules such as student interface, teacher dashboard, and database management.

In the Online Examination Management System, the design helps in organizing features like secure login, role-based access, real-time exam handling, and result generation. It provides a clear blueprint for developers to implement the system efficiently.

Thus, system design acts as a bridge between requirements and implementation, ensuring that the final system meets user expectations and operates effectively.

PROJECT PLANNING & SCHEDULING:

Project planning and scheduling is an important phase in software development that involves organizing tasks, allocating resources, and defining timelines for completing the project. It helps in managing the project efficiently and ensures that all activities are completed within the given time frame.

In the Online Examination Management System, proper planning was carried out before the development phase to clearly define the project scope, objectives, and required resources. The system was designed to provide functionalities such as user authentication, exam creation, question management, exam attempts, and result analysis.

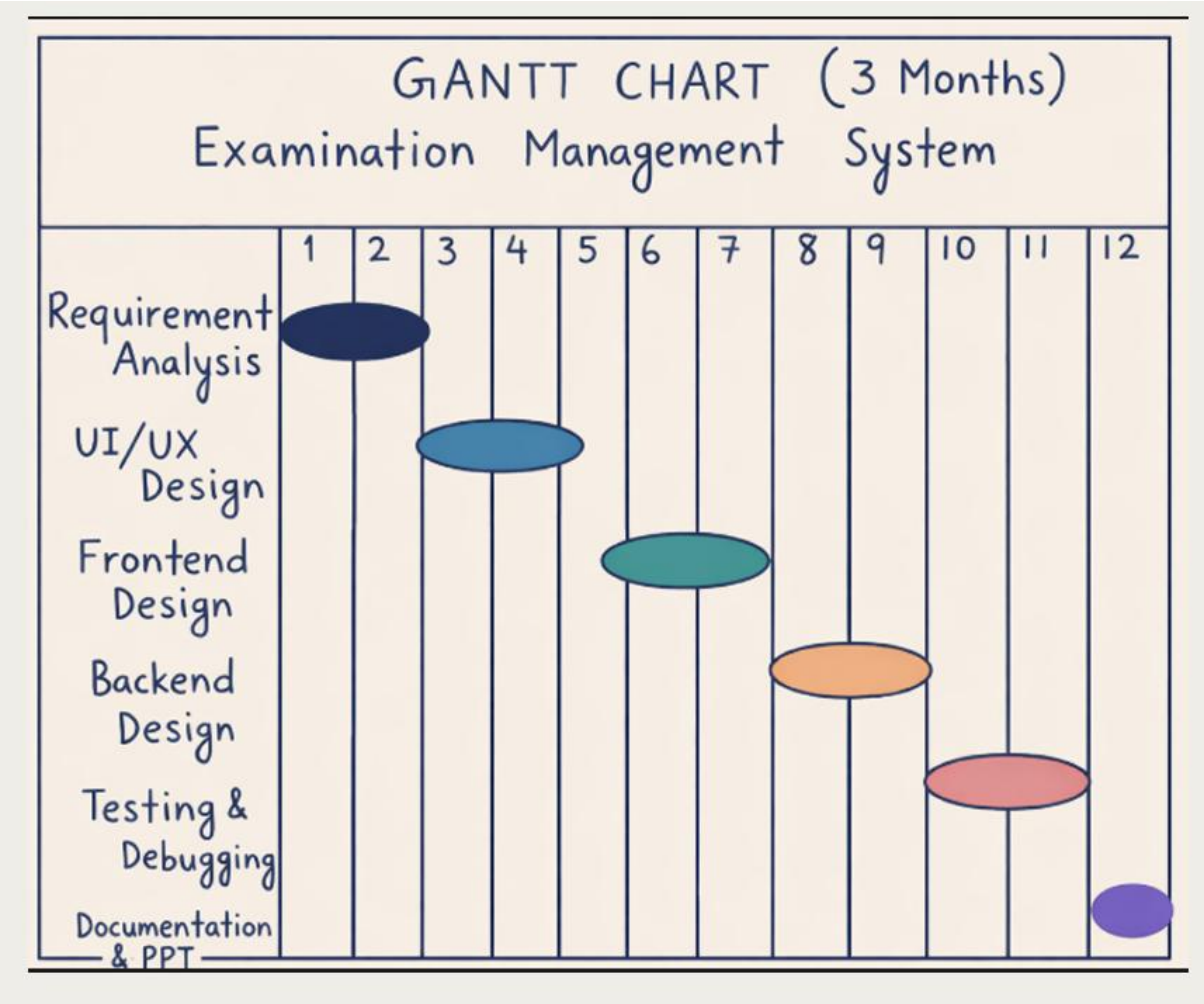
The project was divided into different phases to ensure systematic development. These phases include requirement gathering, system design, development, testing, and implementation. Each phase was assigned a specific time duration to maintain a smooth workflow and avoid delays.

The scheduling of the project was done using tools such as Gantt Chart and PERT Chart. These tools help in visualizing the sequence of tasks, identifying dependencies between different activities, and tracking project progress. They also assist in efficient time management and resource utilization.

Thus, effective planning and scheduling ensured the successful development and timely completion of the Examination Management System.

GANTT CHART:

A Gantt chart is a graphical representation of the project schedule where each task is shown as a bar along a timeline. It helps in understanding the duration of each activity and tracking project progress.



Project Phases and Schedule

Task	Start Date	End Date	Duration
Requirement Analysis	15/12/2025	28/12/2025	2 Weeks
UI/UX Design	29/12/2025	11/01/2026	2 Weeks
Frontend Design	12/01/2026	25/01/2026	2 Weeks
Backend Design	26/01/2026	08/02/2026	2 Weeks
Testing & Debugging	09/02/2026	22/02/2026	2 Weeks
Documentation & PPT	23/02/2026	15/03/2026	3 Weeks

PERT CHART:

PERT **(Program Evaluation and Review Technique)** charts consist of a network of boxes and arrows. The boxes represent activities, and the arrows represent task dependencies. PERT is organized based on events and activities or tasks.

PERT charts are highly useful for complex projects as they help in defining the sequence of tasks and identifying dependencies among them. Through a PERT chart, various task paths are determined, and it becomes easier to understand how different activities are interconnected.

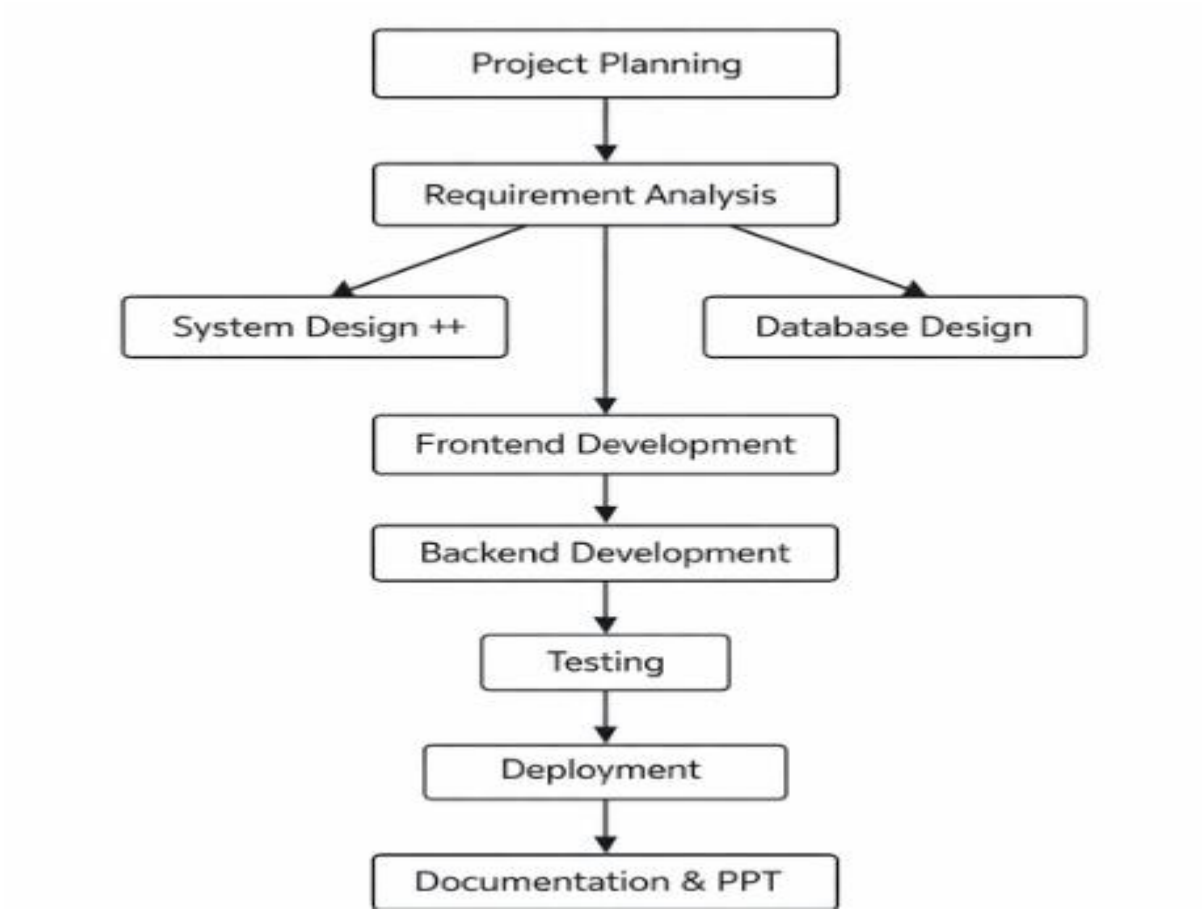
One of the key advantages of PERT is that it enables the calculation of the **critical path**. Each path consists of a combination of tasks that must be completed in sequence. The time and cost associated with each task along a path are analyzed, and the path that requires the maximum time is known as the critical path.

Identifying the critical path allows the project manager to monitor these tasks more closely and allocate resources effectively if any delay occurs. This helps in ensuring that the project is completed on time.

PERT also helps in controlling both time and cost during the project development. It provides flexibility in managing uncertainties and helps in maintaining a balance between completing the project within the scheduled time and within the budget.

Since there can be multiple possible paths depending on task estimations, analyzing the critical path in PERT charts can become complex. However, it provides a detailed and efficient way to manage project execution.

The PERT chart representation of the **Examination Management System** is shown below.



DFD (Data Flow Diagram):

A Data Flow Diagram (DFD) is a graphical representation used in system analysis and design to illustrate how data flows through a system. It shows the movement of data between different components such as input sources, processes, data stores, and output destinations. DFD helps in understanding how information is processed within a system and how different parts of the system interact with each other.

It is one of the most important tools used by system analysts to visualize the logical flow of data without focusing on technical implementation details. The main purpose of a DFD is to provide a clear and structured view of the system, making it easier to identify processes, data transformations, and data storage points.

A DFD consists of four main elements:

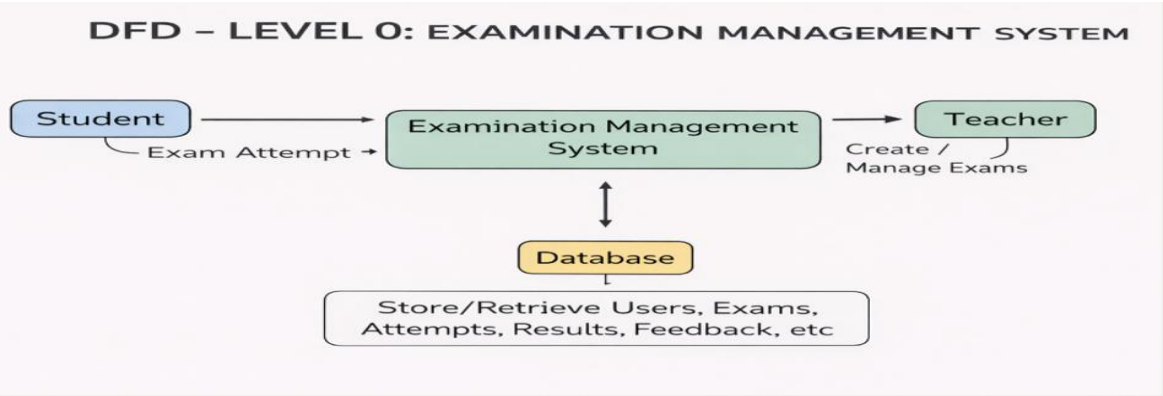
- **External Entities:** These are users or external systems that interact with the system, such as students, teachers, and administrators.
- **Processes:** These represent operations performed on data, such as login verification, exam conduction, and result generation.
- **Data Flows:** These indicate the movement of data between different components of the system.

- **Data Stores:** These represent places where data is stored, such as user database, question bank, and results database.

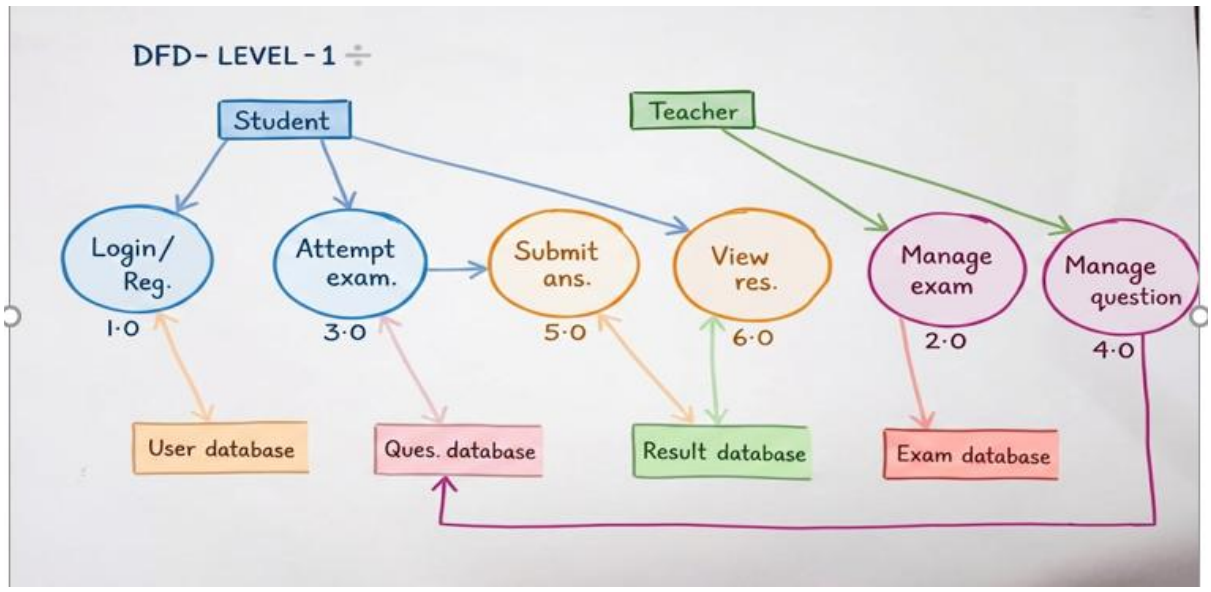
  DFDs are designed in multiple levels such as Level 0, Level 1, and Level 2 to provide different levels of detail. The higher level (Level 0) shows an overall view of the system, while lower levels (Level 1 and Level 2) provide a more detailed breakdown of processes and data flow. This hierarchical approach helps in better understanding, analysis, and communication of system functionality.

  In the context of the **Examination Management System**, the DFD represents how students and teachers interact with the system, how exams are conducted, how answers are submitted, and how results are generated and stored in the database. It ensures clarity, improves system design, and helps in identifying any errors or inefficiencies in the system.

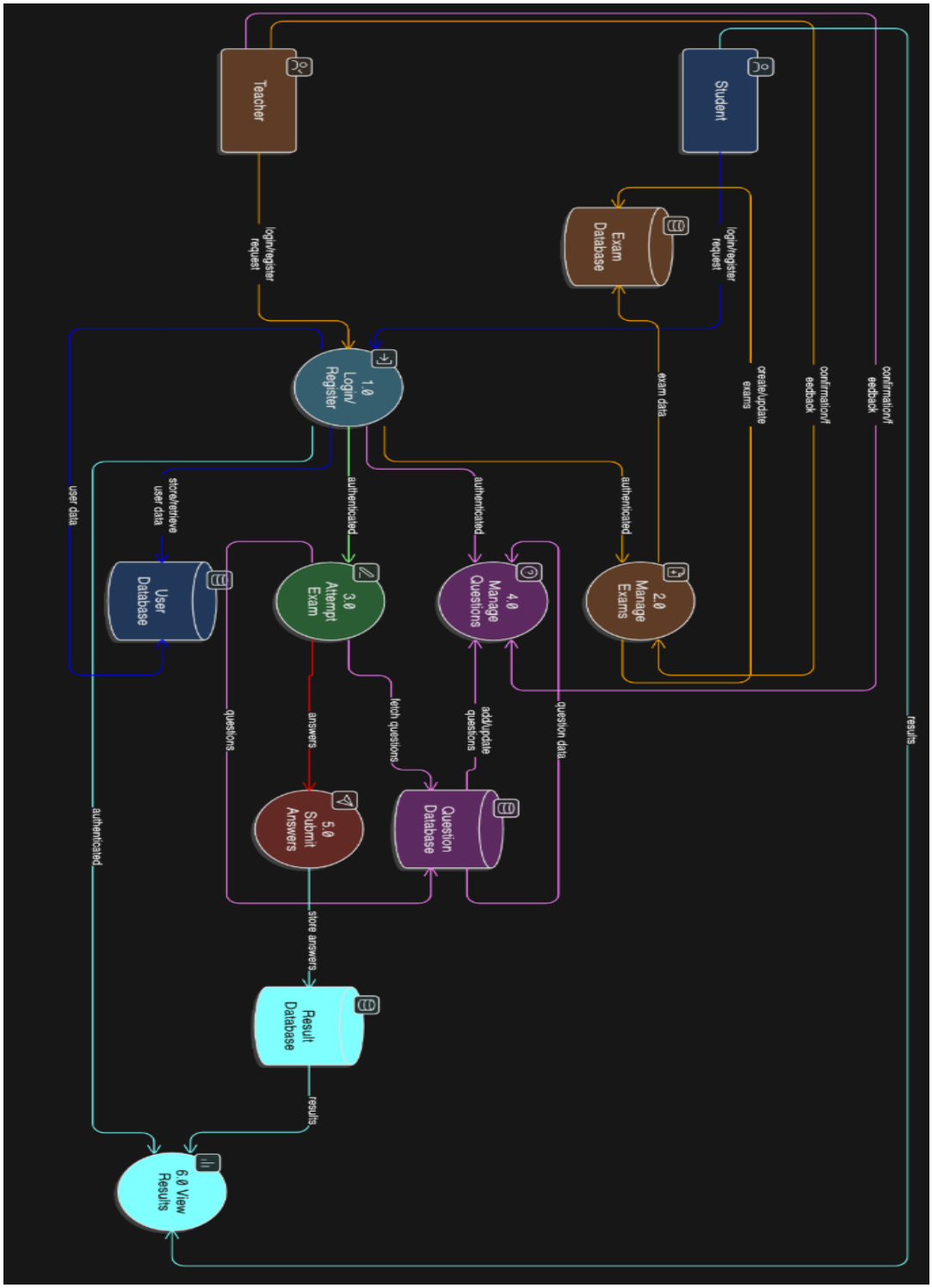
DFD – Level 0:



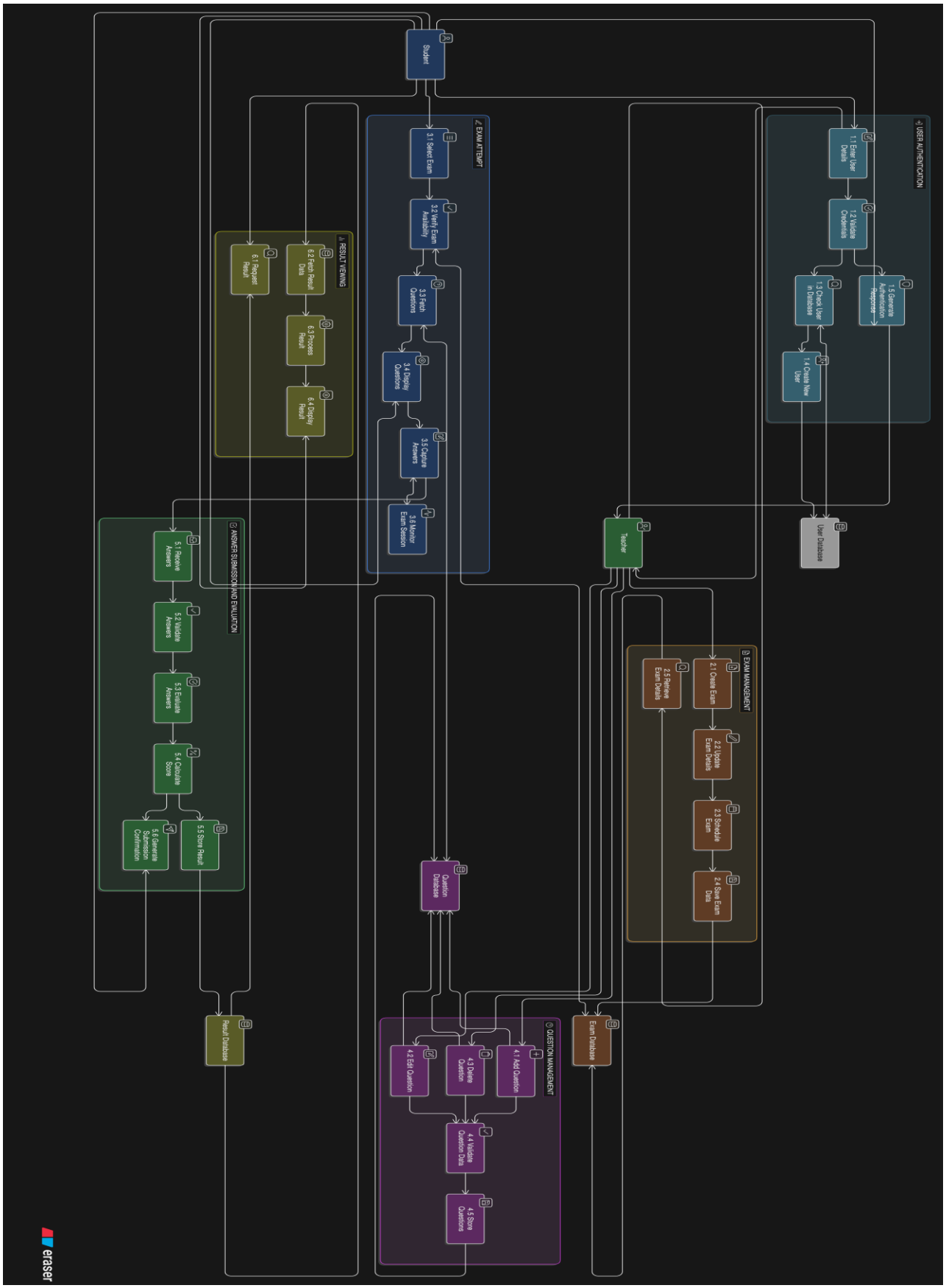
DFD – Level 1:



DFD – Level 2:


Student
Teacher
Exam Database
login/register request
login/register request
1.0 Login/ Register
exam data
create/update exams
confirmation/feedback
confirmation/feedback
2.0 Manage Exams
authenticated
authenticated
authenticated
4.0 Manage Questions
3.0 Attempt Exam
authenticated
store/retrieve user data
User Database
user data
Question Database
add/update questions
question data
fetch questions
answers
questions
5.0 Submit Answers
store answers
Result Database
results
results
6.0 View Results
authenticated

DFD – Level 3:


1. USER AUTHENTICATION
1.1 Enter User Details
1.2 Validate Credentials
1.3 Check User in Database
1.4 Create New User
1.5 Generate Authentication Response
User Database
Student
Teacher
2. EXAM ATTEMPT
3.1 Select Exam
3.2 Verify Exam Availability
3.3 Fetch Questions
3.4 Display Questions
3.5 Capture Answers
3.6 Monitor Exam Session
Question Database
4. QUESTION MANAGEMENT
4.1 Add Question
4.2 Edit Question
4.3 Delete Question
4.4 Validate Question Data
4.5 Store Questions
Exam Database
5. ANSWER SUBMISSION AND EVALUATION
5.1 Receive Answers
5.2 Validate Answers
5.3 Evaluate Answers
5.4 Calculate Score
5.5 Store Result
5.6 Generate Submission Confirmation
Result Database
6. RESULT VIEWING
6.1 Request Result
6.2 Fetch Result Data
6.3 Process Result
6.4 Display Result
3. EXAM MANAGEMENT
2.1 Create Exam
2.2 Update Exam Details
2.3 Schedule Exam
2.4 Save Exam Data
2.5 Retrieve Exam Details
eraser

MODULE DESCRIPTION:

Introduction

The Examination Management System is divided into two main modules: Student Module and Teacher Module. Each module performs specific tasks and together they ensure smooth functioning of the examination process. This modular structure makes the system easy to understand, develop, and manage.

1. Student Module

Description:

The Student Module is responsible for all activities performed by students in the system. It allows students to interact with the system for taking exams and viewing results.

Students can log in using their credentials and access available exams. They can attempt exams within the given time limit and submit their answers. After submission, students can view their results.

The system ensures that once an exam is submitted, students cannot go back and change their answers.

Functions:

- Student login
- Forget password
- Give feedback
- Attempt exam
- Submit answers
- View results

2. Teacher Module

Description:

The Teacher Module is responsible for managing exams and questions. Teachers have full control over exam creation and evaluation.

Teachers can create new exams, add questions, update exam details, and manage question sets. They can also view student performance and results.

This module ensures proper organization and control of the examination process.

Functions:

- Teacher login
- Forget password
- Create exams
- Manage exams
- Manage students
- Add/update/delete questions/delete exam
- View student results

DATA DICTIONERY:

A Data Dictionary is a collection of information about data used in the system. It defines the structure, type, and description of each data element. It helps in understanding how data is stored and used in the system.

User table:

Table Name	Field Name	Data Type	Description
USERS	_id (PK)	ObjectId	System-generated unique identifier for each user
USERS	first_name	String	First name of the user
USERS	last_name	String	Last name of the user
USERS	email	String	Unique email address used for login
USERS	password	String	Encrypted / hashed password of the user
USERS	user_type	String	Role of the user (Student / Teacher)
USERS	date_of_birth	Date	Date of birth of the user
USERS	gender	String	Gender of the user
USERS	institution	String	Name of the institution
USERS	field_of_study	String	Academic field or department
USERS	createdAt	Date	System-generated record creation timestamp
USERS	updatedAt	Date	System-generated record last update timestamp

Exams table:

EXAMS	_id (PK)	ObjectId	System-generated unique identifier for each exam
EXAMS	exam_id	String	Human-readable unique exam code
EXAMS	title	String	Title of the examination
EXAMS	description	String	Detailed description of the exam
EXAMS	target_audience	String	Intended group of students
EXAMS	scheduled_at	Date	Date and time when exam is scheduled
EXAMS	duration_minutes	Number	Total duration of the exam in minutes
EXAMS	status	String	Current exam status (Draft / Published)
EXAMS	created_by (FK)	ObjectId	Teacher (user) who created the exam
EXAMS	createdAt	Date	System-generated exam creation time
EXAMS	updatedAt	Date	System-generated exam update time

Questions table:

Table Name	Field Name	Field Name	Description
QUESTIONS	_id (PK)	ObjectId	System-generated unique identifier for each question
QUESTIONS	exam_id (FK)	String	Exam ID to which the question belongs
QUESTIONS	question_text	String	Text/content of the question
QUESTIONS	question_type	String	Type of question (MCQ / Direct)
QUESTIONS	points	Number	Marks assigned to the question
QUESTIONS	options	Array	List of options for MCQ type questions
QUESTIONS	correct_answer	String	Correct answer for direct questions
QUESTIONS	correct_options	Array	Correct option(s) for MCQ questions
QUESTIONS	attachment	String	Media attachment (audio/video) for future use
QUESTIONS	createdAt	Date	System-generated question creation time
QUESTIONS	updatedAt	Date	System-generated question update time

Attempts table:

ATTEMPTS	_id (PK)	ObjectId	System-generated unique identifier for each attempt
ATTEMPTS	exam_id (FK)	String	Exam ID attempted by the student
ATTEMPTS	student_id (FK)	ObjectId	Student who attempted the exam
ATTEMPTS	start_time	Date	Time when the exam attempt started
ATTEMPTS	end_time	Date	Time when the exam attempt ended
ATTEMPTS	score	Number	Total marks obtained by the student
ATTEMPTS	answers	Array	List of answers submitted by the student
ATTEMPTS	ip_address	String	IP address of the student during exam
ATTEMPTS	coordinates	GeoPoint	Geographical location of the student
ATTEMPTS	completed	Boolean	Indicates whether exam attempt is completed
ATTEMPTS	createdAt	Date	System-generated attempt creation time
ATTEMPTS	updatedAt	Date	System-generated attempt update time

Feedback table:

FEEDBACK	_id (PK)	ObjectId	System-generated unique identifier for feedback
FEEDBACK	name	String	Name of the user submitting feedback
FEEDBACK	email	String	Email address of the feedback provider
FEEDBACK	message	String	Feedback message or suggestion
FEEDBACK	createdAt	Date	System-generated feedback submission time
FEEDBACK	updatedAt	Date	System-generated feedback update time

Passwordresets table:

Table Name	Field Name	Data Type	Description
PASSWORD_RESETS	_id (PK)	ObjectId	System-generated unique identifier for each password reset request
PASSWORD_RESETS	email	String	Email address of the user requesting password reset
PASSWORD_RESETS	otp	String	One-Time Password sent to the user for verification
PASSWORD_RESETS	expiry	Date	Expiration time of the OTP
PASSWORD_RESETS	isVerified	Boolean	Indicates whether OTP has been verified or not
PASSWORD_RESETS	createdAt	Date	System-generated record creation timestamp
PASSWORD_RESETS	updatedAt	Date	System-generated record last update timestamp

ENTITY RELATIONSHIP DIAGRAM (ER DIAGRAM):

Introduction

An Entity Relationship (ER) Diagram is a graphical representation used in system design to describe the structure of a database. It shows how different entities in a system are related to each other. ER diagrams help in understanding the data flow, relationships, and organization of information in a clear and structured way.

In the **Examination Management System**, the ER diagram represents how data such as users, exams, questions, attempts, and results are interconnected within the system.

Purpose of ER Diagram

The main purpose of an ER diagram is:

- To visualize the database structure
- To identify entities and their attributes
- To represent relationships between entities
- To simplify database design and development
- To improve communication between developers and users

Entities in the System

The major entities used in this system are:

1. Users

This entity stores information about all users of the system, including students and teachers. It contains details like name, email, password, and user type.

2. Exams

This entity represents the exams created by teachers. It includes details such as exam title, description, schedule, and duration.

3. Questions

This entity stores all questions related to exams. Each question belongs to a specific exam and includes question type, options, and correct answers.

4. Attempts

This entity records the attempts made by students during exams. It stores information such as start time, end time, answers, and score.

5. Feedback

This entity stores feedback provided by users, including their name, email, and message.

6. Password Reset

This entity manages password reset requests using OTP verification.

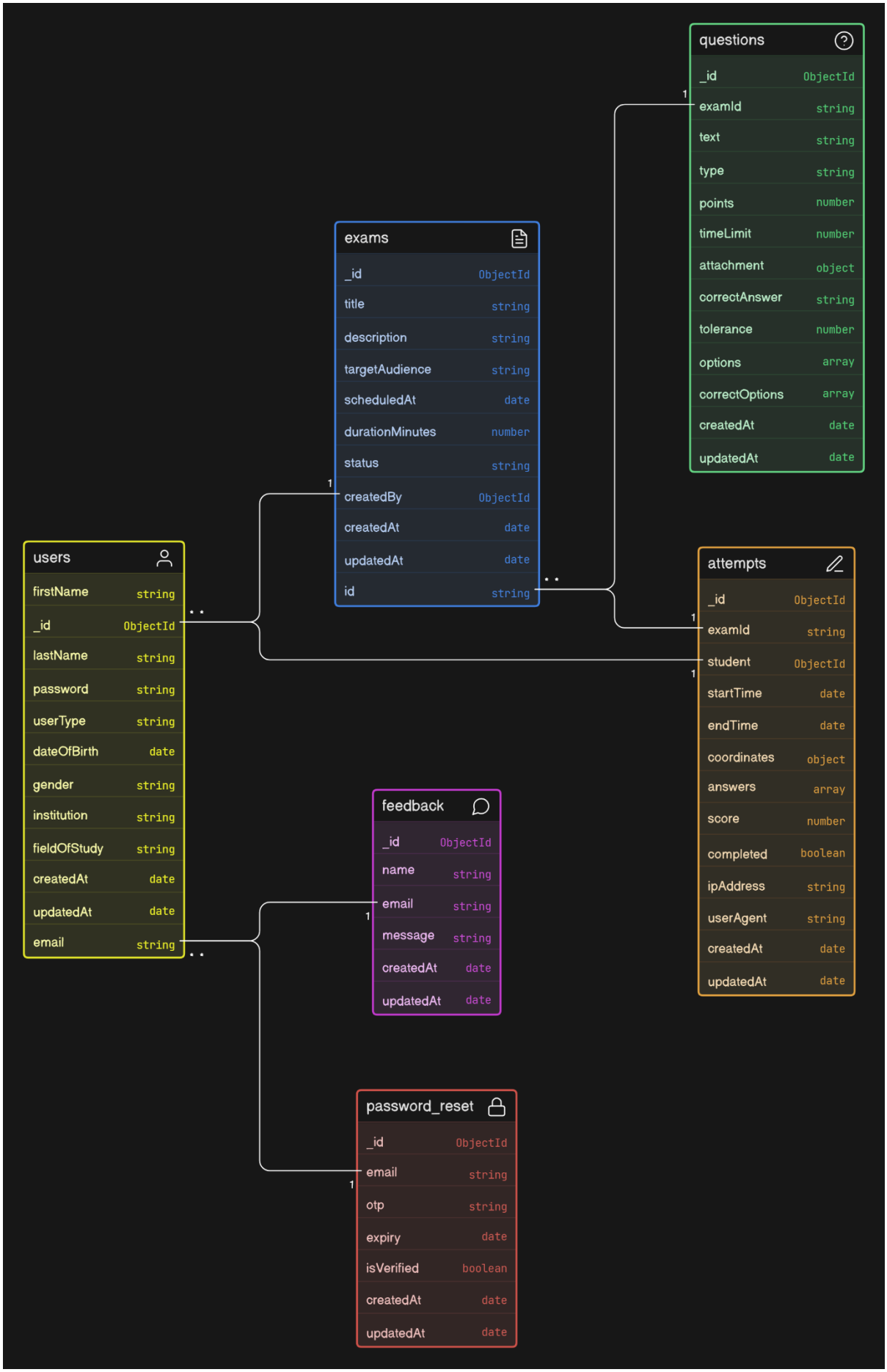
Relationships Between Entities

The relationships between entities in the system are as follows:

- A **user (teacher)** creates multiple exams
- An **exam** contains multiple questions
- A **user (student)** attempts exams
- Each **attempt** is associated with a specific exam
- Each **attempt** stores answers and generates a score
- Users can submit feedback
- Users can request password reset

Features of ER Diagram

- Uses entities, attributes, and relationships
- Provides a clear structure of the database
- Helps in avoiding data redundancy
- Improves data consistency and integrity
- Supports better system planning


questions
_id ObjectId
examId string
text string
type string
points number
timeLimit number
attachment object
correctAnswer string
tolerance number
options array
correctOptions array
createdAt date
updatedAt date
exams
_id ObjectId
title string
description string
targetAudience string
scheduledAt date
durationMinutes number
status string
createdBy ObjectId
createdAt date
updatedAt date
id string
users
firstName string
_id ObjectId
lastName string
password string
userType string
dateOfBirth date
gender string
institution string
fieldOfStudy string
createdAt date
updatedAt date
email string
attempts
_id ObjectId
examId string
student ObjectId
startTime date
endTime date
coordinates object
answers array
score number
completed boolean
ipAddress string
userAgent string
createdAt date
updatedAt date
feedback
_id ObjectId
name string
email string
message string
createdAt date
updatedAt date
password_reset
_id ObjectId
email string
otp string
expiry date
isVerified boolean
createdAt date
updatedAt date

SCOPE AND FUTURE ENHANCEMENT:

Scope:

The Examination Management System has a wide scope in the field of digital education and online examination. It provides an efficient and automated way to conduct exams, manage student data, and evaluate results.

The system can be used by schools, colleges, and educational institutions to reduce manual work and improve accuracy in examination processes. It allows students to appear for exams from anywhere, which increases flexibility and convenience.

The system ensures secure handling of exam data, user authentication, and result generation. It also provides real-time evaluation and instant result display, making the process faster and more efficient.

This project is scalable and can be implemented in both small institutions and large organizations. It supports proper data storage, easy access, and efficient management of examinations.

Future Enhancements

Although the system is fully functional, it can be further enhanced by adding advanced features:

- **Mobile Application**
  Develop a mobile app (Android/iOS) for better accessibility and user experience.
- **Online Proctoring System**
  Add AI-based monitoring such as face detection, eye tracking, and voice detection to prevent cheating.

- **Multi-language Support**
  Provide support for multiple languages to make the system user-friendly.

- **Question Randomization**
  Randomize questions and options to reduce cheating during exams.

IMPLEMENTATION METHODOLOGY:

Implementation methodology refers to the systematic process of developing, testing, and deploying the system. It ensures that the Examination Management System is built efficiently and works correctly in real-world conditions.

In this project, a step-by-step approach is followed to implement the Examination Management System.

1. Requirement Analysis

In this phase, the requirements of the system are collected and analyzed. It includes understanding the needs of students and teachers, defining system functionalities, and determining the scope of the project.

2. System Design

In this phase, the overall structure of the system is designed. It includes:

- Database design (Users, Exams, Questions, Attempts, Results, etc.)
- User Interface design (Student and Teacher panels)
- Data Flow Diagrams (DFD) and system architecture

3. Development

In this phase, the actual coding of the system is done.

- Frontend is developed using **HTML, CSS, and JavaScript**
- Backend is developed using **Node.js and Express.js**
- Database is managed using **MongoDB (Mongoose)**

4. Testing

After development, the system is tested to ensure proper functionality.

- Login and authentication testing
- Exam attempt and submission testing
- Result generation testing
- Database operations testing

Errors are identified and fixed in this phase.

5. Implementation (Deployment)

The system is deployed using a local development server (Node.js environment). Users (students and teachers) can access the system through a web browser.

6. Maintenance

After deployment, the system is regularly monitored and maintained. Bugs are fixed, performance is improved, and new features are added when required.

TESTING:

Testing is the process of evaluating a system to ensure that it works correctly and meets the specified requirements. It helps in identifying errors, bugs, and issues before the system is deployed.

In the **Examination Management System**, testing is performed to ensure that all functionalities such as login, exam management, question handling, exam attempts, and result generation work properly and securely.

Software testing consists of the following stages:

a) TEST PLAN DEVELOPMENT

Software testing is a critical part of software quality assurance. It represents the final review of specifications, design, and coding. The main objective of testing is to identify errors systematically with minimum effort and time, thereby improving the reliability of the system.

b) TESTING STRATEGY

A testing strategy defines how testing will be carried out to detect maximum errors efficiently. It ensures that different modules of the Examination Management System are tested properly and allows flexibility for modifications during development.

c) UNIT TESTING

Unit testing focuses on testing individual modules of the system.

In this project, the following modules are tested separately:

- Login/Register module
- Exam management module
- Question management module
- Attempt exam module
- Result module

Each module is tested using normal and boundary input values. Errors found during testing are fixed before moving to the next phase.

d) INTEGRATION TESTING

Integration testing is performed after combining all modules into a complete system. It checks whether the modules interact correctly with each other.

For example:

- Student login → attempt exam → submit answers → view result
- Teacher login → create exam → add questions

Any errors during module interaction are identified and resolved.

e) VALIDATION TESTING

Validation testing ensures that the system meets all user requirements and works as expected.

Different test cases, including normal and random inputs, are used to verify system performance. It confirms that the system behaves correctly under all conditions.

TEST CASE DESIGN

The following techniques are used to design effective test cases:

1. CONDITION TESTING

This technique tests logical conditions within the system.

Example:

- Valid login credentials → successful login
- Invalid credentials → error message "Invalid username or password"

2. BOUNDARY VALUE ANALYSIS

This technique tests values at the boundary limits.

Example:

- Minimum and maximum exam duration
- Attempting submission after time limit → system should restrict submission

3. EQUIVALENCE PARTITIONING

This method divides input data into valid and invalid groups to reduce test cases.

Example:

- Valid input → correct login, successful exam attempt
- Invalid input → empty fields, wrong credentials → error message

LIMITATIONS AND FURTHER ENHANCEMENTS:

Limitations of the System

Although the Examination Management System works efficiently, it has some limitations:

- The system requires a stable internet connection to operate
- Security features are basic and may not be sufficient for large-scale examination environments
- The user interface is simple and can be further improved
- Limited support for handling a very large number of students simultaneously
- No dedicated mobile application support
- No advanced monitoring features for preventing cheating during exams

Further Enhancements

The system can be improved in the future by adding the following features:

1. Advanced Security Features

Implement OTP-based login and two-factor authentication to enhance system security.

2. Mobile Application Development

Develop Android/iOS applications for better accessibility and user experience.

3. Online Proctoring System

Integrate features like face detection, eye tracking, and voice monitoring to prevent cheating during exams.

4. Advanced Reporting System

Generate detailed reports and graphical analysis of student performance.

SECURITY MECHANISM:

Security is one of the most important aspects of any Examination Management System. The system ensures that user data, exam content, and results are protected from unauthorized access and misuse. Various security mechanisms are implemented to maintain confidentiality, integrity, and availability of data.

1. User Authentication

The system uses a secure login mechanism where students and teachers must enter valid credentials (email/username and password) to access the system. Only authenticated users are allowed to perform operations.

2. Password Protection

Passwords are securely stored using encryption techniques. Strong password rules are applied to ensure better security and prevent unauthorized access.

3. Authorization

Different users have different access levels:

- **Student** → Can attempt exams and view results
- **Teacher** → Can create exams, manage questions, and view student performance

This ensures that only authorized users can access specific features of the system.

5. Database Security

Data stored in the database is protected using secure queries and proper validation techniques. This helps in preventing attacks such as injection attacks and unauthorized data access.

5. Input Validation

All user inputs are validated to ensure that invalid or harmful data is not entered into the system. This improves system reliability and security.

6. OTP-Based Password Recovery

The system includes an OTP-based password reset mechanism. When a user requests a password reset, an OTP is sent to their registered email, which must be verified before changing the password.

7. Error Handling

The system avoids displaying sensitive error messages to users. This prevents attackers from gaining useful information about the system.

DATABASE MODEL:

Introduction

The database model defines how data is stored, managed, and accessed within the system. In the Examination Management System, a modern web-based architecture is used, where the frontend, backend, and database are separated for better performance and scalability.

Three-Tier Architecture

The Examination Management System follows a **three-tier architecture**, which consists of:

1. Presentation Layer (Frontend)

This layer is responsible for user interaction. It includes the user interface developed using HTML, CSS, and JavaScript.
Students and teachers interact with the system through this layer to perform operations such as login, exam attempt, and result viewing.

2. Application Layer (Backend)

This layer handles the business logic of the system. It is developed using Node.js and Express.js.
It processes user requests, performs validation, manages authentication, and communicates with the database.

3. Data Layer (Database)

This layer is responsible for storing and managing data. The system uses **MongoDB**, a NoSQL database, to store data in the form of documents.

MongoDB provides flexibility, scalability, and efficient handling of large amounts of data. Each collection stores related data such as users, exams, questions, and attempts.

Database Design (MongoDB Collections)

The system uses the following main collections:

- **Users Collection** → Stores student and teacher details
- **Exams Collection** → Stores exam information
- **Questions Collection** → Stores questions related to exams
- **Attempts Collection** → Stores student responses and scores
- **Feedback Collection** → Stores user feedback
- **Password Reset Collection** → Manages OTP-based password reset

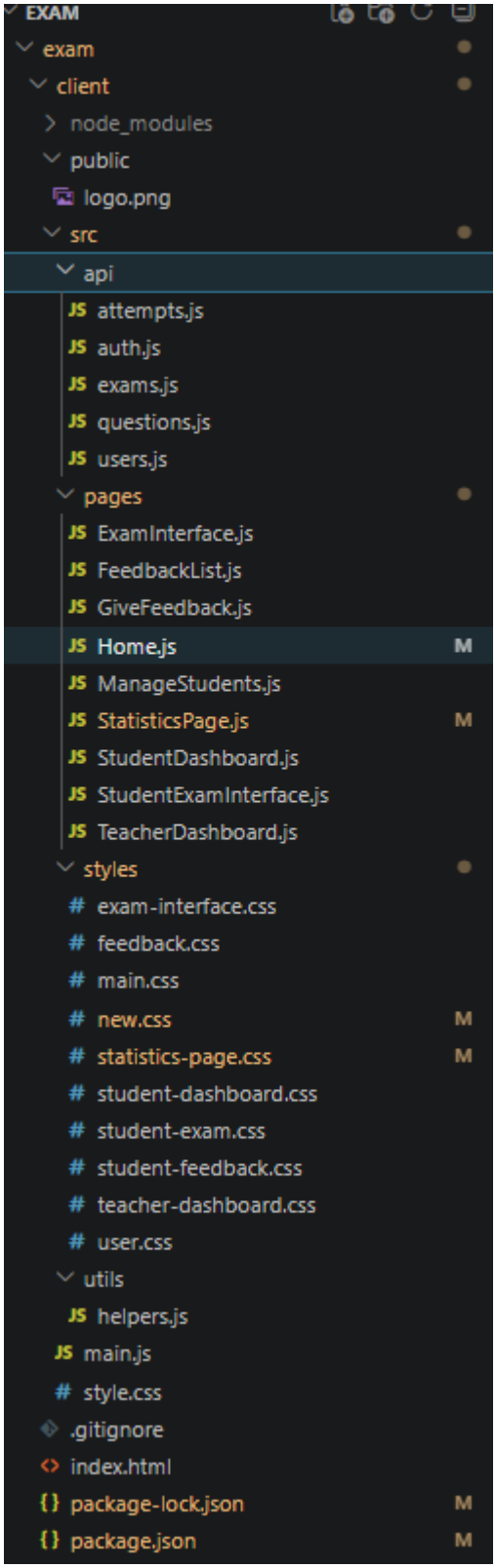
---

Advantages of the Database Model

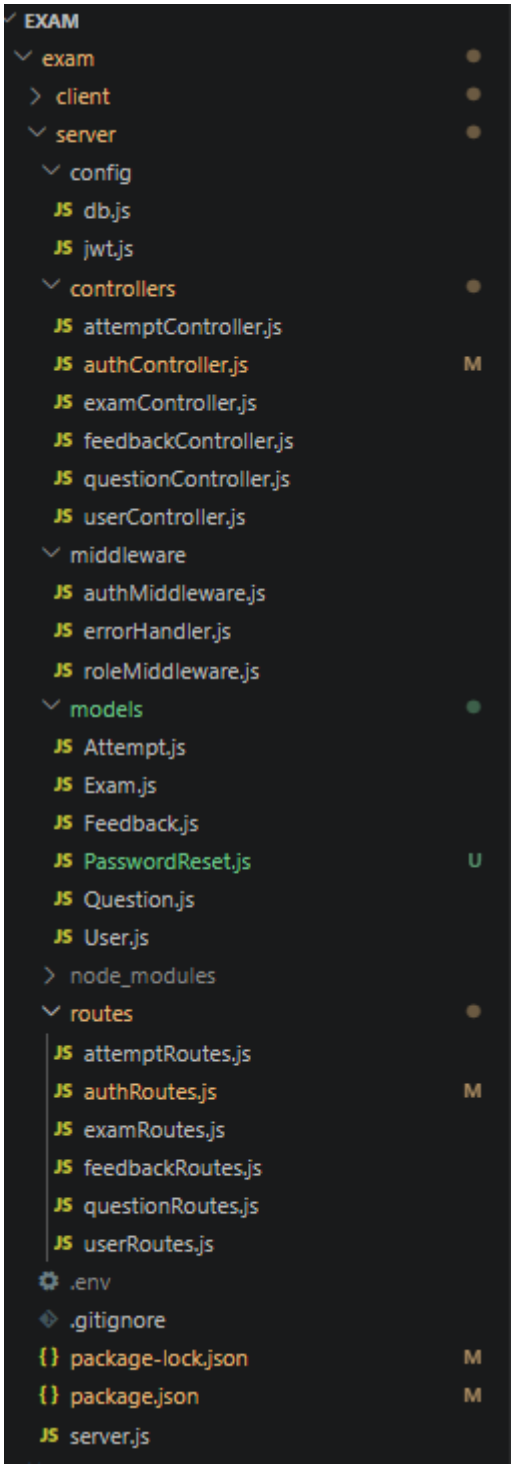
- Provides separation of concerns (frontend, backend, database)
- Improves system scalability and performance
- Ensures better data management and security
- Supports real-time data access and updates
- Flexible schema design using MongoDB

Structure of the Project (Code part):

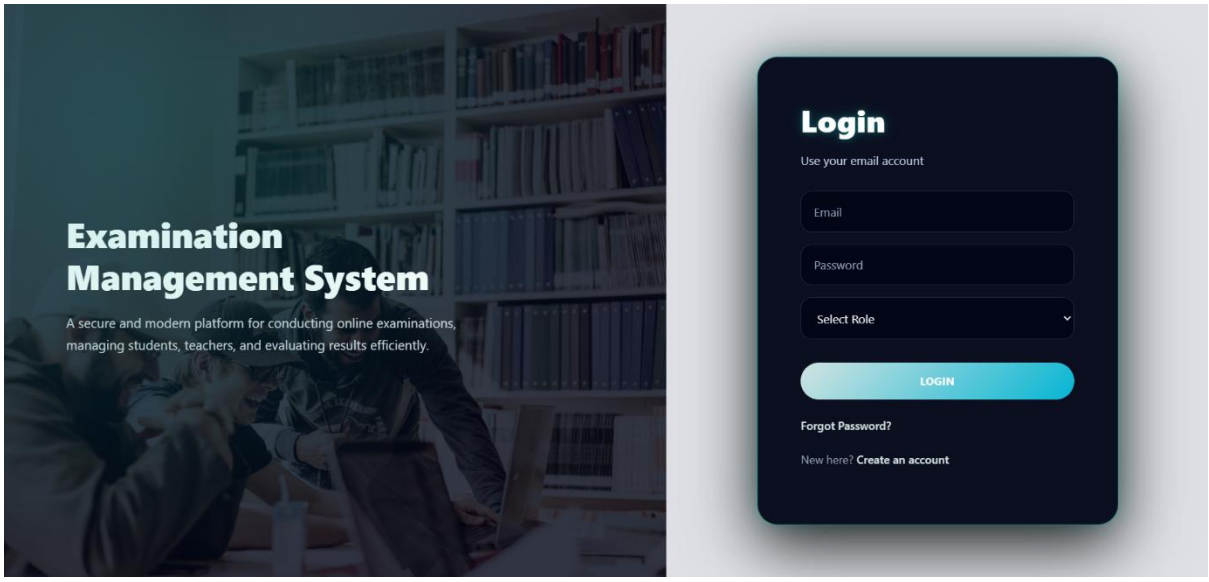
Client Side



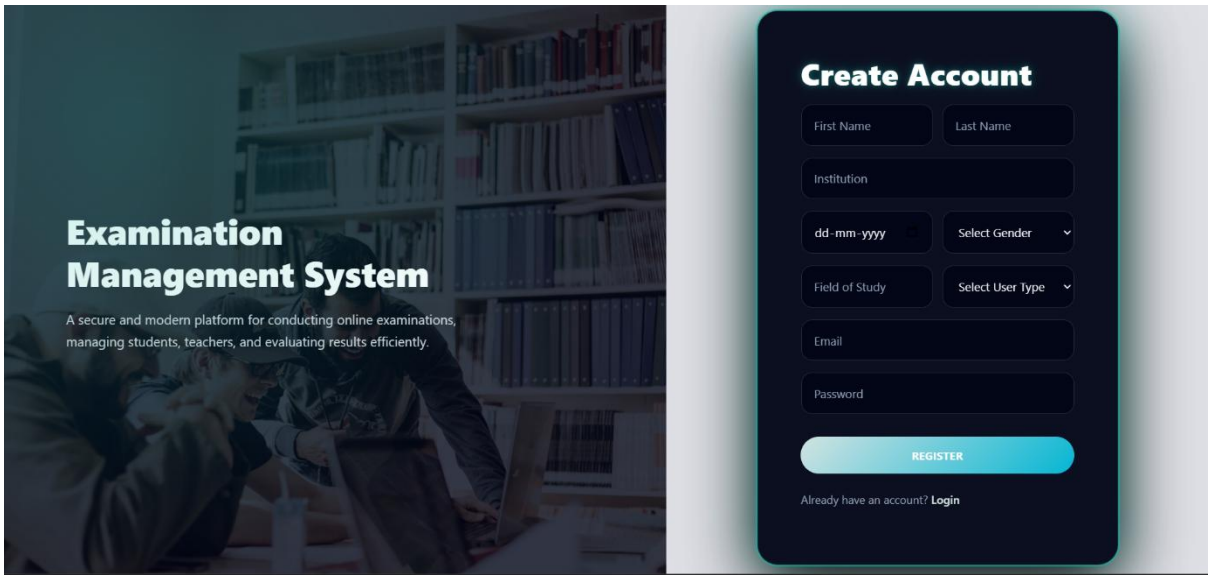
Server Side



Snapshot of Student Login Page / Teacher Login page:



Snapshot of User Account Creation Page:



Index.html Page: This file is the main entry point of the Examination Management System web application. It provides the basic structure of the webpage and loads all the necessary frontend components.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/ico+xml" href="/logo.png"
/>
    <meta name="viewport" content="width=device-width,
initial-scale=1.0" />

    <title>Examination Management System</title>
  </head>
  <body>
    <div id="app"></div>
    <script type="module" src="/src/main.js"></script>
  </body>
</html>
```

Home.js :- This code creates login and create account page

```
import "../styles/new.css";
import { login, register } from "../api/auth.js";

export function renderHome() {
  const app = document.getElementById("app");

  app.innerHTML = `
    <div class="auth-wrapper">

      <!-- LEFT -->
      <div class="auth-left">
        <div class="auth-left-content">
          <h1>Examination Management System</h1>
          <p>
            A secure and modern platform for conducting
online examinations,
            managing students, teachers, and evaluating
results efficiently.
          </p>
```

```
        </div>
      </div>

      <!-- RIGHT -->
      <div class="auth-right">

        <!-- LOGIN -->
        <div class="auth-box" id="login-box">
          <h2>Login</h2>
          <p class="sub-text">Use your email account</p>

          <form id="login-form">
            <input type="email" name="email"
placeholder="Email" required />
            <input type="password" name="password"
placeholder="Password" required />

            <select name="role" required>
              <option value="" disabled selected>Select
Role</option>
              <option value="student">Student</option>
              <option value="teacher">Teacher</option>
            </select>

            <button type="submit">LOGIN</button>
          </form>

          <p class="switch-text">
            <span id="forgot-password-link">Forgot
Password?</span>
          </p>

          <p class="switch-text">
            New here?
            <span id="show-register">Create an
account</span>
          </p>
        </div>
```

```
        <!-- REGISTER -->
        <div class="auth-box hidden" id="register-box">
          <h2>Create Account</h2>

          <form id="register-form">
            <div class="row">
              <input name="firstName" placeholder="First
Name" required />
              <input name="lastName" placeholder="Last Name"
required />
            </div>

            <input name="institution"
placeholder="Institution" required />

            <div class="row">
              <input type="date" name="dateOfBirth" required
/>
              <select name="gender" required>
                <option value="" disabled selected>Select
Gender</option>
                <option value="male">Male</option>
                <option value="female">Female</option>
              </select>
            </div>

            <div class="row">
              <input name="fieldOfStudy" placeholder="Field
of Study" required />
              <select name="userType" required>
                <option value="" disabled selected>Select
User Type</option>
                <option value="student">Student</option>
              <!--  <option value="teacher">Teacher</option>
-->
              </select>
            </div>
```

```
            <input type="email" name="email"
placeholder="Email" required />
            <input type="password" name="password"
placeholder="Password" required />

            <button type="submit">REGISTER</button>
          </form>

          <p class="switch-text">
            Already have an account?
            <span id="show-login">Login</span>
          </p>
        </div>

      </div>
    </div>
  `;

  // toggle
  document.getElementById("show-register").onclick = () => {
    document.getElementById("login-
box").classList.add("hidden");
    document.getElementById("register-
box").classList.remove("hidden");
  };

  document.getElementById("show-login").onclick = () => {
    document.getElementById("register-
box").classList.add("hidden");
    document.getElementById("login-
box").classList.remove("hidden");
  };

  // login
  document.getElementById("login-form").onsubmit = async (e)
=> {
    e.preventDefault();
```

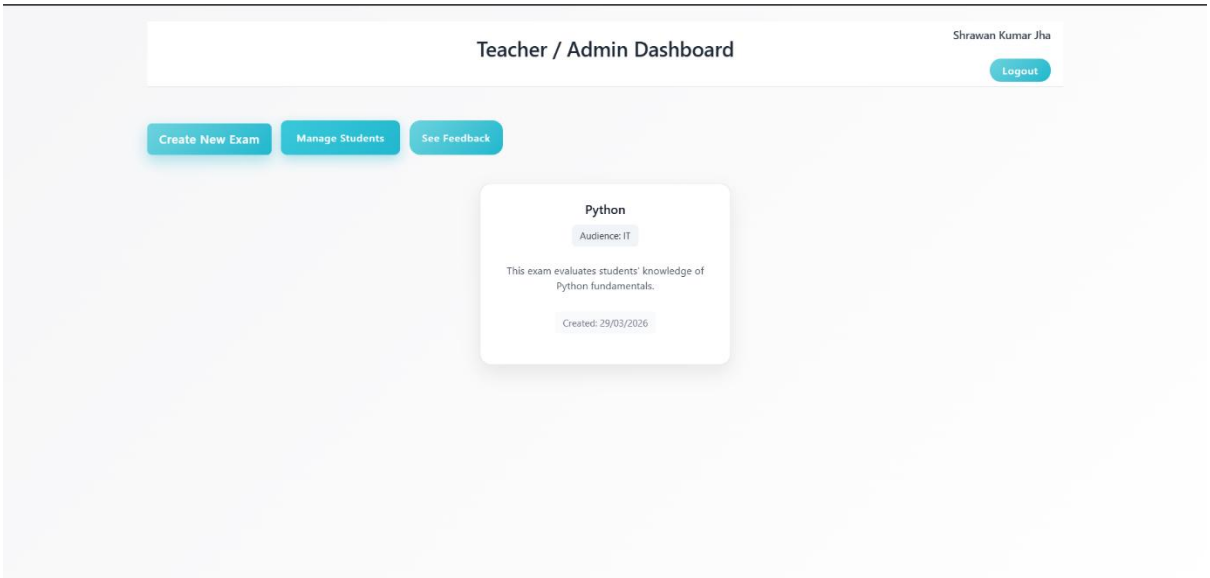
```
    const f = e.target;

    await login({
      email: f.email.value,
      password: f.password.value,
      role: f.role.value,
    });
  };

  // register
  document.getElementById("register-form").onsubmit = async
(e) => {
    e.preventDefault();
    const f = e.target;

    await register({
      firstName: f.firstName.value,
      lastName: f.lastName.value,
      institution: f.institution.value,
      dateOfBirth: f.dateOfBirth.value,
      gender: f.gender.value,
      fieldOfStudy: f.fieldOfStudy.value,
      email: f.email.value,
      password: f.password.value,
      userType: f.userType.value,
    });
  };

  document.getElementById("forgot-password-link").onclick =
() => {
    renderForgotPassword();
  };
}

// =======================
// 📧 FORGOT PASSWORD (CENTER)
// =======================
function renderForgotPassword() {
```

```
  const app = document.getElementById("app");

  app.innerHTML = `
    <div class="center-screen">
      <div class="auth-box">
        <h2>Forgot Password</h2>
        <input type="email" id="email" placeholder="Enter
your email" />
        <button id="sendOtp">Send OTP</button>
        <p id="back">Back to Login</p>
      </div>
    </div>
  `;

  document.getElementById("sendOtp").onclick = async () => {
    const email = document.getElementById("email").value;

    const res = await
fetch("http://localhost:3000/api/auth/forgot-password", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({ email }),
    });

    const data = await res.json();
    alert(data.message);

    if (res.ok) renderVerifyOtp(email);
  };

  document.getElementById("back").onclick = renderHome;
}

// =======================
// 1️⃣2️⃣3️⃣4️⃣ VERIFY OTP (CENTER)
// =======================
```

```
function renderVerifyOtp(email) {
  const app = document.getElementById("app");

  app.innerHTML = `
    <div class="center-screen">
      <div class="auth-box">
        <h2>Verify OTP</h2>
        <input type="text" id="otp" placeholder="Enter OTP"
/>
        <button id="verify">Verify</button>
      </div>
    </div>
  `;

  document.getElementById("verify").onclick = async () => {
    const otp = document.getElementById("otp").value;

    const res = await
fetch("http://localhost:3000/api/auth/verify-otp", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({ email, otp }),
    });

    const data = await res.json();
    alert(data.message);

    if (res.ok) renderResetPassword(email);
  };
}

// =======================
// 🔐 RESET PASSWORD (CENTER)
// =======================
function renderResetPassword(email) {
  const app = document.getElementById("app");
```

```
  app.innerHTML = `
    <div class="center-screen">
      <div class="auth-box">
        <h2>Reset Password</h2>
        <input type="password" id="newPass" placeholder="New
Password" />
        <button id="reset">Update Password</button>
      </div>
    </div>
  `;

  document.getElementById("reset").onclick = async () => {
    const newPassword =
document.getElementById("newPass").value;

    const res = await
fetch("http://localhost:3000/api/auth/reset-password", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({ email, newPassword }),
    });

    const data = await res.json();
    alert(data.message);

    if (res.ok) renderHome();
  };
}
```

Snapshot of Teacher dashboard:



TeacherDashboard.js

```
import { fetchExams, createExam } from "../api/exams.js";
import { logout, getLoggedInUser } from "../api/auth.js";
import { generateUniqueId } from "../utils/helpers.js";

export async function renderTeacherDashboard() {
  const app = document.getElementById("app");

  // 🔐 AUTH CHECK (TEACHER ONLY)
  const user = getLoggedInUser();
  if (!user || user.userType !== "teacher") {
    history.pushState(null, "", "/");
    window.dispatchEvent(new Event("popstate"));
    return;
  }

  // ======================
  // MAIN LAYOUT
  // ======================
  app.innerHTML = `
    <section class="dashboard teacher-dashboard">
```

```
      <!-- HEADER -->
      <header class="dashboard-header">
        <h1>Teacher / Admin Dashboard</h1>

        <div class="header-right">
          <span class="user-name">${user.name}</span>
          <button id="logout-btn" class="secondary-
btn">Logout</button>
        </div>
      </header>

      <!-- ACTION BAR -->
      <div class="dashboard-actions">
        <button id="create-exam-btn" class="primary-btn">
          Create New Exam
        </button>

        <button id="manage-students-btn" class="secondary-
btn">
          Manage Students
        </button>

        <button id="see-feedback-btn" class="secondary-btn">
          See Feedback
        </button>
      </div>

      <!-- CONTENT -->
      <div id="dashboard-content">
        <div class="loading">Loading exams...</div>
      </div>

      <!-- CREATE EXAM MODAL -->
      <div id="create-exam-dialog" class="dialog-overlay
hidden">
        <div class="dialog-content">
          <h2>Create New Exam</h2>
```

```
          <form id="create-exam-form">
            <div class="form-group">
              <label>Title</label>
              <input type="text" name="title" required />
            </div>

            <div class="form-group">
              <label>Description</label>
              <textarea name="description" rows="3"
required></textarea>
            </div>

            <div class="form-group">
              <label>Target Audience</label>
              <input type="text" name="targetAudience"
required />
            </div>

            <div id="exam-error" class="form-error
hidden"></div>

            <div class="form-actions">
              <button type="button" id="cancel-create-exam"
class="secondary-btn">
                Cancel
              </button>
              <button type="submit" class="primary-btn">
                Create Exam
              </button>
            </div>
          </form>
        </div>
      </div>

    </section>
  `;
```

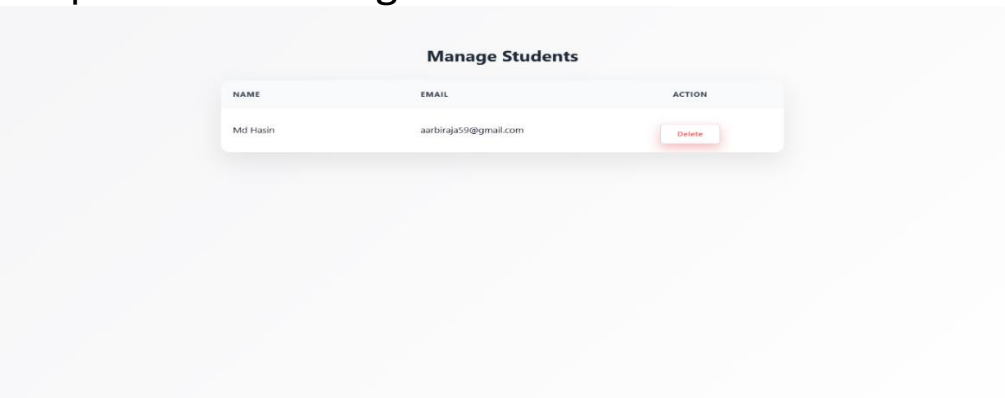
```
  const content = document.getElementById("dashboard-
content");
  const dialog = document.getElementById("create-exam-
dialog");
  const form = document.getElementById("create-exam-form");
  const errorBox = document.getElementById("exam-error");

  // =======================
  // LOAD EXAMS
  // =======================
  async function loadExams() {
    content.innerHTML = `<div class="loading">Loading
exams...</div>`;
    try {
      const data = await fetchExams();
      const exams = Array.isArray(data) ? data : data?.exams
|| [];
      renderExamCards(exams);
    } catch {
      content.innerHTML = `
        <div class="error-message">
          Failed to load exams
        </div>
      `;
    }
  }

  // =======================
  // CREATE EXAM MODAL
  // =======================
  document.getElementById("create-exam-btn").onclick = () =>
    dialog.classList.remove("hidden");

  document.getElementById("cancel-create-exam").onclick = ()
=>
    dialog.classList.add("hidden");

  // =======================
```

```
  // SUBMIT EXAM
  // ======================
  form.addEventListener("submit", async (e) => {
    e.preventDefault();
    errorBox.classList.add("hidden");

    const examData = {
      id: generateUniqueId(),
      title: form.title.value.trim(),
      description: form.description.value.trim(),
      targetAudience: form.targetAudience.value.trim(),
      createdAt: new Date().toISOString(),
    };

    try {
      await createExam(examData);
      dialog.classList.add("hidden");
      form.reset();
      loadExams();
    } catch (err) {
      errorBox.textContent = err.message || "Failed to
create exam";
      errorBox.classList.remove("hidden");
    }
  });

  // ======================
  // MANAGE STUDENTS
  // ======================
  document.getElementById("manage-students-btn").onclick =
() => {
    history.pushState(null, "", "/teacher/students");
    window.dispatchEvent(new Event("popstate"));
  };

  // ======================
  // SEE FEEDBACK
  // ======================
```

```
  document.getElementById("see-feedback-btn").onclick = ()
=> {
    history.pushState(null, "", "/teacher/feedbacks");
    window.dispatchEvent(new Event("popstate"));
  };

  // ======================
  // LOGOUT
  // ======================
  document.getElementById("logout-btn").onclick = () => {
    logout();
    history.pushState(null, "", "/");
    window.dispatchEvent(new Event("popstate"));
  };

  loadExams();
}

/* ======================
   EXAM CARDS
====================== */
function renderExamCards(exams) {
  const container = document.getElementById("dashboard-
content");

  if (!exams.length) {
    container.innerHTML = `
      <div class="no-exams-message">
        <p>No exams created yet.</p>
        <p>Click "Create New Exam" to add one.</p>
      </div>
    `;
    return;
  }

  container.innerHTML = `
    <div class="exam-cards-container">
      ${exams
```

```
        .map(
          (e) => `
        <div class="exam-card" data-id="${e.id}">
          <h3 class="exam-title">${e.title}</h3>
          <div class="exam-audience">
            Audience: ${e.targetAudience}
          </div>
          <p class="exam-description">${e.description}</p>
          <div class="exam-meta">
            Created: ${new
Date(e.createdAt).toLocaleDateString()}
          </div>
        </div>
      `
        )
        .join("")}
    </div>
  `;

  container.querySelectorAll(".exam-card").forEach((card) =>
{
    card.onclick = () => {
      history.pushState(null, "",
`/teacher/exams/${card.dataset.id}`);
      window.dispatchEvent(new Event("popstate"));
    };
  });
}
```

Snapshot of manage Student dashboard:



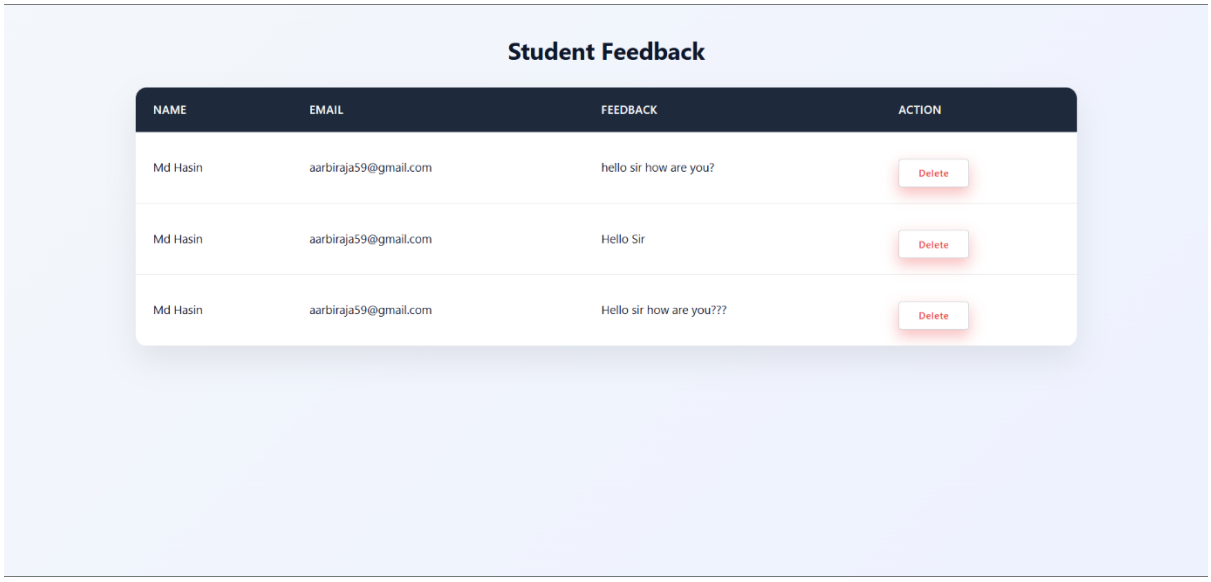
ManageStudent.js:

```
import { fetchStudents, deleteStudent } from
"../api/users.js";
import { getLoggedInUser } from "../api/auth.js";

export async function renderManageStudents() {
  const app = document.getElementById("app");

  // 🔐 AUTH CHECK
  const user = getLoggedInUser();
  if (!user || user.userType !== "teacher") {
    history.pushState(null, "", "/");
    window.dispatchEvent(new Event("popstate"));
    return;
  }

  app.innerHTML = `<h2>Loading students...</h2>`;

  try {
    const res = await fetchStudents();
    const students = res.students || [];

    app.innerHTML = `
      <section class="manage-students-page">
        <h1>Manage Students</h1>

        ${
          students.length === 0
            ? `<p class="no-students">No students
found.</p>`
            : `
        <table class="students-table">
          <thead>
            <tr>
              <th>Name</th>
              <th>Email</th>
              <th>Action</th>
            </tr>
```

```
          </thead>
          <tbody>
            ${students
              .map(
                (s) => `
              <tr>
                <td>${s.firstName} ${s.lastName}</td>
                <td>${s.email}</td>
                <td>
                  <button class="delete-btn" data-
id="${s._id}">
                    Delete
                  </button>
                </td>
              </tr>
            `
              )
              .join("")}
          </tbody>
        </table>
        `
        }
      </section>
    `;

    // 🗑 DELETE HANDLER
    document.querySelectorAll(".delete-btn").forEach((btn)
=> {
      btn.onclick = async () => {
        const confirmDelete = confirm(
          "Are you sure you want to delete this student?"
        );
        if (!confirmDelete) return;

        try {
          await deleteStudent(btn.dataset.id);
          renderManageStudents(); // 🔄 refresh list
        } catch (err) {
```

```
          alert("Failed to delete student");
        }
      };
    });
  } catch (err) {
    console.error(err);
    app.innerHTML = `<p>Error loading students</p>`;
  }
}
```

Snapshot of manage see Feedback:



FeedbackList.js

```
import { getLoggedInUser } from "../api/auth.js";
import "../styles/feedback.css";

export async function renderFeedbackList() {
  const app = document.getElementById("app");
  const user = getLoggedInUser();

  // 🔐 TEACHER ONLY ACCESS
  if (!user || user.userType !== "teacher") {
    history.pushState(null, "", "/");
```

```
    window.dispatchEvent(new Event("popstate"));
    return;
  }

  // ========================
  // PAGE LAYOUT
  // ========================
  app.innerHTML = `
  <section class="feedback-page">
    <h1 class="page-title">Student Feedback</h1>

    <div class="table-wrapper">
      <table class="feedback-table">
        <thead>
          <tr>
            <th>Name</th>
            <th>Email</th>
            <th>Feedback</th>
            <th>Action</th>
          </tr>
        </thead>
        <tbody id="feedback-container">
          <tr>
            <td colspan="4" class="loading">Loading
feedback...</td>
          </tr>
        </tbody>
      </table>
    </div>
  </section>
`;


  const container = document.getElementById("feedback-
container");

  // ========================
  // FETCH FEEDBACK
  // ========================
```

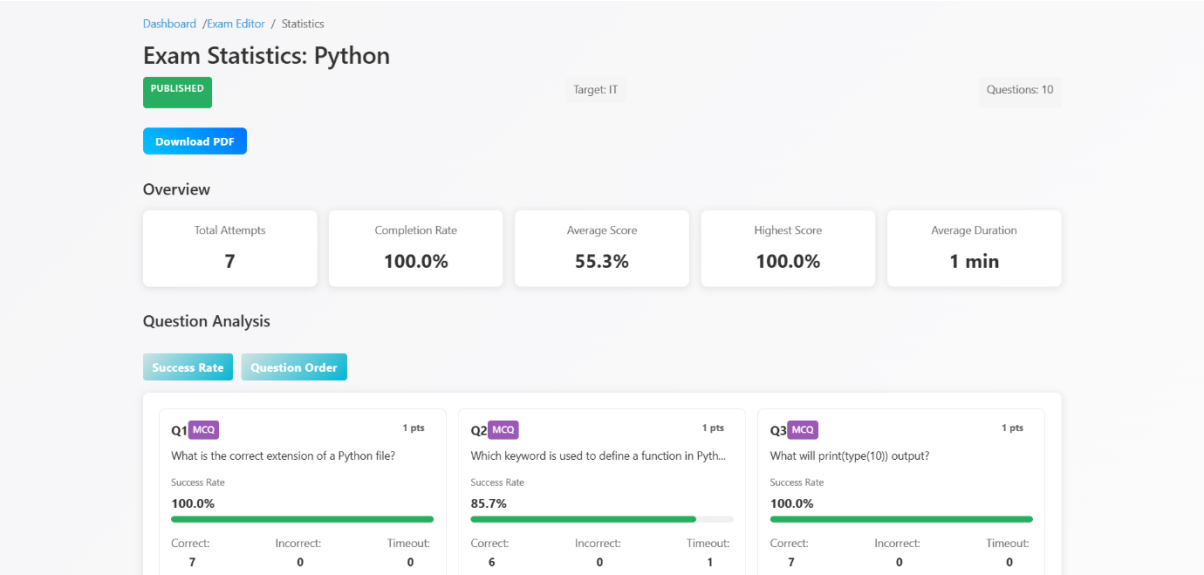
```
  try {
    const res = await
fetch("http://localhost:3000/api/feedback", {
      method: "GET",
      headers: {
        Authorization: `Bearer
${localStorage.getItem("token")}`,
      },
    });

    const result = await res.json();

    if (!res.ok) {
      throw new Error(result.message || "Failed to fetch
feedback");
    }

    // Support both formats:
    // 1) [ {...}, {...} ]
    // 2) { success: true, data: [...] }
    const feedbacks = Array.isArray(result)
      ? result
      : result.data || [];

    renderFeedbackCards(feedbacks);
  } catch (err) {
    console.error("Feedback fetch error:", err);
    container.innerHTML = `
      <p class="error">❌ ${err.message || "Error loading
feedback"}</p>
    `;
  }
}

// ========================
// RENDER FEEDBACK CARDS
// ========================
function renderFeedbackCards(feedbacks) {
```

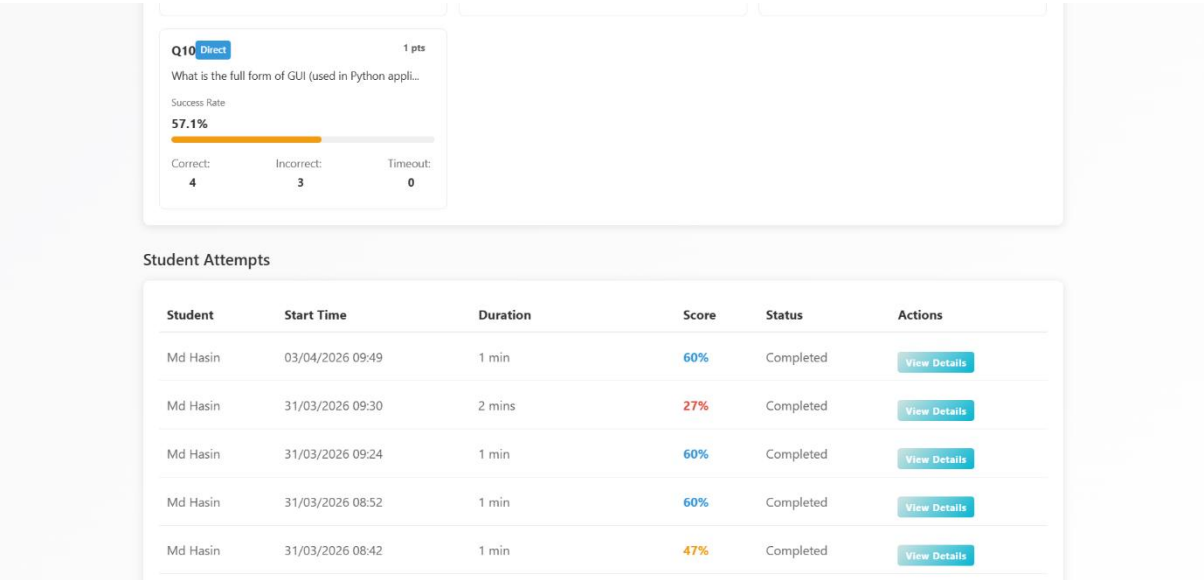
```
  const container = document.getElementById("feedback-
container");

  if (!feedbacks.length) {
    container.innerHTML = `
      <tr>
        <td colspan="4" class="empty">No feedback submitted
yet.</td>
      </tr>
    `;
    return;
  }

  container.innerHTML = feedbacks
    .map(
      (f) => `
        <tr>
          <td>${escapeHTML(f.name)}</td>
          <td>${escapeHTML(f.email)}</td>
          <td class="message-
cell">${escapeHTML(f.message)}</td>
          <td>
            <button
              class="delete-btn"
              onclick="deleteFeedback('${f._id}')"
            >
              Delete
            </button>
          </td>
        </tr>
      `
    )
    .join("");
}

// ========================
// DELETE FEEDBACK (GLOBAL)
// ========================
```

```
window.deleteFeedback = async function (id) {
  if (!confirm("Are you sure you want to delete this
feedback?")) return;

  try {
    const res = await
fetch(`http://localhost:3000/api/feedback/${id}`, {
      method: "DELETE",
      headers: {
        Authorization: `Bearer
${localStorage.getItem("token")}`,
      },
    });

    const result = await res.json();

    if (!res.ok) {
      throw new Error(result.message || "Delete failed");
    }

    // reload list
    renderFeedbackList();
  } catch (err) {
    console.error("Delete feedback error:", err);
    alert(`❌ ${err.message || "Failed to delete
feedback"}`);
  }
};

// ==========================
// BASIC HTML ESCAPE (SECURITY)
// ==========================
function escapeHTML(str = "") {
  return String(str).replace(/[&<>"']/g, (m) => {
    return {
      "&": "&amp;",
      "<": "&lt;",
      ">": "&gt;",
```

```
      '"': "&quot;",
      "'": "&#39;",
    }[m];
  });
}
```

Snapshot of manage Statistic Page:





StatisticsPage.js

```
import { fetchExamById } from "../api/exams.js";
import { fetchExamAttempts, fetchExamStatistics } from
"../api/attempts.js";
import "../styles/statistics-page.css";
import jsPDF from "jspdf";
import html2canvas from "html2canvas";

/**
 * Renders the exam statistics page
 * @param {string} examId - The ID of the exam to show
statistics for
 */
export async function renderStatisticsPage(examId) {
  const app = document.getElementById("app");
  app.innerHTML = `
    <div class="statistics-page">
      <div class="loading">Loading statistics...</div>
    </div>
  `;

  try {
    // Fetch exam details, attempts, and statistics in
parallel
    const [examResponse, attemptsResponse,
statisticsResponse] =
      await Promise.all([
        fetchExamById(examId),
        fetchExamAttempts(examId),
        fetchExamStatistics(examId),
      ]);

    // Extract the exam object from the response
    const exam = examResponse.exam || examResponse;

    // Extract the attempts array from the response
    const attempts = attemptsResponse.attempts ||
attemptsResponse;
```

```
    // Extract the statistics object from the response
    const statistics = statisticsResponse.statistics ||
statisticsResponse;

    // Render the statistics page
    renderStatisticsContent(app, exam, statistics,
attempts);
  } catch (error) {
    console.error("Error loading statistics:", error);
    app.querySelector(".statistics-page").innerHTML = `
      <div class="error-message">
        <h3>Error Loading Statistics</h3>
        <p>${
          error.message ||
          "Failed to load exam statistics. Please try again
later."
        }</p>
        <button id="back-btn" class="secondary-btn">Back to
Exam</button>
      </div>
    `;
    document.getElementById("back-
btn").addEventListener("click", () => {
      history.pushState(null, "",
`/teacher/exams/${examId}`);
      window.dispatchEvent(new Event("popstate"));
    });
  }
}

/**
 * Renders the statistics content
 * @param {HTMLElement} app - The app container element
 * @param {Object} exam - The exam object
 * @param {Object} statistics - The statistics from the
backend
 * @param {Array} attempts - Array of attempt objects
```

```
 */
function renderStatisticsContent(app, exam, statistics,
attempts) {
  app.innerHTML = `
    <div class="statistics-page" id="reportContent">
      <header class="stats-header">
        <div class="breadcrumb">
          <a href="/teacher/exams" id="back-to-dashboard"
data-link="spa">Dashboard</a> /
          <a href="/teacher/exams/${
            exam.id || exam._id
          }" id="back-to-exam" data-link="spa">Exam
Editor</a> /
          <span>Statistics</span>
        </div>
        <h1>Exam Statistics: ${exam.title}</h1>
        <p class="exam-meta">
          <span class="exam-status
${exam.status}">${exam.status}</span>
          <span class="exam-audience">Target:
${exam.targetAudience}</span>
          <span class="exam-questions">Questions: ${
            exam.questions.length
          }</span>
        </p>
        <button id="downloadPdf" class="pdf-btn">Download
PDF</button>
      </header>

      <main class="stats-content">
        ${statistics.totalAttempts === 0 ?
renderNoAttemptsMessage() : ""}

        <section class="stats-overview">
          <h2>Overview</h2>
          <div class="stats-cards">
            <div class="stats-card">
```

```
              <div class="stats-card-header">Total
Attempts</div>
              <div class="stats-card-
value">${statistics.totalAttempts}</div>
            </div>
            <div class="stats-card">
              <div class="stats-card-header">Completion
Rate</div>
              <div class="stats-card-
value">${statistics.completionRate.toFixed(
                1
              )}%</div>
            </div>
            <div class="stats-card">
              <div class="stats-card-header">Average
Score</div>
              <div class="stats-card-
value">${statistics.averageScore.toFixed(
                1
              )}%</div>
            </div>
            <div class="stats-card">
              <div class="stats-card-header">Highest
Score</div>
              <div class="stats-card-
value">${statistics.highestScore.toFixed(
                1
              )}%</div>
            </div>
            <div class="stats-card">
              <div class="stats-card-header">Average
Duration</div>
              <div class="stats-card-
value">${formatDuration(
                statistics.averageDuration
              )}</div>
            </div>
          </div>
```

```
        </section>

        ${
          statistics.totalAttempts > 0
            ? `

          <section class="stats-questions">
            <h2>Question Analysis</h2>
            <div class="question-filters">
              <button class="filter-btn active" data-
filter="success-rate">Success Rate</button>
              <button class="filter-btn" data-
filter="question-order">Question Order</button>
            </div>
            <div class="question-stats-container">
              ${renderQuestionStats(statistics.questionStats
, exam.questions)}
            </div>
          </section>

          <section class="stats-attempts">
            <h2>Student Attempts</h2>
            <div class="attempts-table-container">
              <table class="attempts-table">
                <thead>
                  <tr>
                    <th>Student</th>
                    <th>Start Time</th>
                    <th>Duration</th>
                    <th>Score</th>
                    <th>Status</th>
                    <th>Actions</th>
                  </tr>
                </thead>
                <tbody>
                  ${renderAttemptsTable(attempts)}
                </tbody>
              </table>
```

```
            </div>
          </section>
        `
            : ""
        }
      </main>
    </div>
  `;

  // Set up event listeners
  setupEventListeners(exam);
  setupPdfDownload();
}

/**
 * Renders the score distribution chart
 * @param {Object} statistics - The statistics object
 * @returns {string} - HTML for the score distribution chart
 */
function renderScoreDistribution(statistics) {
  // Calculate the counts for each score range
  const excellentCount = statistics.scores
    ? statistics.scores.filter((score) => score >=
80).length
    : 0;
  const goodCount = statistics.scores
    ? statistics.scores.filter((score) => score >= 60 &&
score < 80).length
    : 0;
  const averageCount = statistics.scores
    ? statistics.scores.filter((score) => score >= 40 &&
score < 60).length
    : 0;
  const poorCount = statistics.scores
    ? statistics.scores.filter((score) => score < 40).length
    : 0;
```

```
  const totalCount = statistics.completedAttempts ||
statistics.totalAttempts;

  return `
    <div class="chart-bar excellent" style="height:
${getPercentage(
      excellentCount,
      totalCount
    )}%">
      <span class="bar-value">${excellentCount}</span>
    </div>
    <div class="chart-bar good" style="height:
${getPercentage(
      goodCount,
      totalCount
    )}%">
      <span class="bar-value">${goodCount}</span>
    </div>
    <div class="chart-bar average" style="height:
${getPercentage(
      averageCount,
      totalCount
    )}%">
      <span class="bar-value">${averageCount}</span>
    </div>
    <div class="chart-bar poor" style="height:
${getPercentage(
      poorCount,
      totalCount
    )}%">
      <span class="bar-value">${poorCount}</span>
    </div>
  `;
}

/**
 * Renders the question statistics
```

```
 * @param {Array} questionStats - Array of question
statistics from the backend
 * @param {Array} questions - Array of exam questions
 * @returns {string} - HTML string for question statistics
 */
function renderQuestionStats(questionStats, questions) {
  if (!questionStats || questionStats.length === 0) {
    return '<div class="no-data">No question data
available</div>';
  }

  return `
    <div class="question-stats-grid">
      ${questionStats
        .map((stat) => {
          // Find the corresponding question to get
additional details
          const question = questions.find(
            (q) =>
              q._id === stat.questionId ||
              q.questionNumber === stat.questionNumber
          );

          return `
          <div class="question-stat-card" data-question-
id="${stat.questionId}">
            <div class="question-stat-header">
              <span class="question-
number">Q${stat.questionNumber}</span>
              <span class="question-type ${stat.type}">${
            stat.type === "direct" ? "Direct" : "MCQ"
          }</span>
              <span class="question-points">${
                question ? question.points || 1 : 1
              } pts</span>
            </div>
            <div class="question-
text">${truncateText(stat.text, 100)}</div>
```

```
            <div class="question-stat-metrics">
              <div class="metric success-rate">
                <div class="metric-label">Success Rate</div>
                <div class="metric-
value">${stat.successRate.toFixed(1)}%</div>
                <div class="progress-bar">
                  <div class="progress-fill
${getSuccessRateClass(
                    stat.successRate
                  )}" style="width:
${stat.successRate}%"></div>
                </div>
              </div>
              <!--
              <div class="metric time-spent">
                <div class="metric-label">Avg. Time</div>
                <div class="metric-value">${formatSeconds(
                  stat.averageTime || 0
                )}</div>
                <div class="time-indicator">
                  <div class="time-fill" style="width:
${getTimePercentage(
                    stat.averageTime || 0,
                    question ? question.timeLimit || 60 : 60
                  )}%"></div>
                </div>
              </div> --!>
            </div>
            <div class="question-stat-details">
              <div class="detail-item">
                <span class="detail-label">Correct:</span>
                <span class="detail-
value">${stat.correctCount}</span>
              </div>
              <div class="detail-item">
                <span class="detail-label">Incorrect:</span>
                <span class="detail-
value">${stat.incorrectCount}</span>
```

```
              </div>
              <div class="detail-item">
                <span class="detail-label">Timeout:</span>
                <span class="detail-
value">${stat.timeoutCount}</span>
              </div>
            </div>
          </div>
        `;
        })
        .join("")}
    </div>
  `;
}

/**
 * Renders a message when there are no attempts
 * @returns {string} - HTML string for no attempts message
 */
function renderNoAttemptsMessage() {
  return `
    <div class="no-attempts-message">
      <div class="message-icon">📊</div>
      <h3>No Exam Attempts Yet</h3>
      <p>Statistics will be available once students start
taking the exam.</p>
    </div>
  `;
}

/**
 * Renders the attempts table
 * @param {Array} attempts - Array of attempt objects
 * @returns {string} - HTML string for attempts table
 */
function renderAttemptsTable(attempts) {
  if (!attempts || attempts.length === 0) {
```

```
    return '<tr><td colspan="6" class="no-data">No attempts
recorded</td></tr>';
  }

  // Sort attempts by start time (newest first)
  const sortedAttempts = [...attempts].sort(
    (a, b) => new Date(b.startTime) - new Date(a.startTime)
  );

  return sortedAttempts
    .map((attempt) => {
      const student = attempt.student || {
        firstName: "Unknown",
        lastName: "Student",
      };
      const startTime = new Date(attempt.startTime);
      const endTime = attempt.endTime ? new
Date(attempt.endTime) : null;
      const duration = endTime ? (endTime - startTime) /
(1000 * 60) : null; // in minutes

      return `
      <tr class="${attempt.completed ? "" : "in-progress"}">
        <td>${student.firstName} ${student.lastName}</td>
        <td>${formatDate(startTime)}</td>
        <td>${duration ? formatDuration(duration) : "In
progress"}</td>
        <td class="${getScoreClass(attempt.score)}">${
        attempt.completed ? `${attempt.score}%` : "-"
      }</td>
        <td>${attempt.completed ? "Completed" : "In
progress"}</td>
        <td>
          <button class="view-attempt-btn" data-attempt-
id="${
            attempt._id
          }">View Details</button>
        </td>
```

```
      </tr>
    `;
    })
    .join("");
}

/**
 * Set up event listeners for the statistics page
 * @param {Object} exam - The exam object
 */
function setupEventListeners(exam) {
  // Back to dashboard
  const backToDashboard = document.getElementById("back-to-
dashboard");
  if (backToDashboard) {
    backToDashboard.addEventListener("click", (e) => {
      e.preventDefault();
      history.pushState(null, "", "/teacher/exams");
      window.dispatchEvent(new Event("popstate"));
    });
  }

  // Back to exam
  const backToExam = document.getElementById("back-to-
exam");
  if (backToExam) {
    backToExam.addEventListener("click", (e) => {
      e.preventDefault();
      history.pushState(null, "", `/teacher/exams/${exam.id
|| exam._id}`);
      window.dispatchEvent(new Event("popstate"));
    });
  }

  // Question filters
  const filterButtons = document.querySelectorAll(".filter-
btn");
  if (filterButtons.length > 0) {
```

```
    filterButtons.forEach((btn) => {
      btn.addEventListener("click", () => {
        // Remove active class from all buttons
        filterButtons.forEach((b) =>
b.classList.remove("active"));

        // Add active class to clicked button
        btn.classList.add("active");

        // Get the filter type
        const filterType = btn.dataset.filter;

        // Apply the filter
        applyQuestionFilter(filterType);
      });
    });
  }

  // View attempt details
  const viewAttemptButtons =
document.querySelectorAll(".view-attempt-btn");
  if (viewAttemptButtons.length > 0) {
    viewAttemptButtons.forEach((btn) => {
      btn.addEventListener("click", () => {
        const attemptId = btn.dataset.attemptId;
        showAttemptDetailsModal(attemptId);
      });
    });
  }
}

/**
 * Apply a filter to the question statistics
 * @param {string} filterType - The type of filter to apply
 */
function applyQuestionFilter(filterType) {
  const container = document.querySelector(".question-stats-
grid");
```

```
  if (!container) return;

  const cards =
Array.from(container.querySelectorAll(".question-stat-
card"));

  // Sort the cards based on the filter type
  let sortedCards;

  switch (filterType) {
    case "success-rate":
      sortedCards = cards.sort((a, b) => {
        const aRate = parseFloat(
          a.querySelector(".success-rate .metric-
value").textContent
        );
        const bRate = parseFloat(
          b.querySelector(".success-rate .metric-
value").textContent
        );
        return bRate - aRate; // Highest first
      });
      break;
    case "time-spent":
      sortedCards = cards.sort((a, b) => {
        const aTime = parseTimeString(
          a.querySelector(".time-spent .metric-
value").textContent
        );
        const bTime = parseTimeString(
          b.querySelector(".time-spent .metric-
value").textContent
        );
        return bTime - aTime; // Longest first
      });
      break;
    case "question-order":
      sortedCards = cards.sort((a, b) => {
```

```
        const aNum = parseInt(
          a.querySelector(".question-
number").textContent.substring(1)
        );
        const bNum = parseInt(
          b.querySelector(".question-
number").textContent.substring(1)
        );
        return aNum - bNum; // Ascending order
      });
      break;
    default:
      sortedCards = cards;
  }

  // Remove all cards from the container
  cards.forEach((card) => card.remove());

  // Add the sorted cards back to the container
  sortedCards.forEach((card) =>
container.appendChild(card));

  // Add animation class
  container.classList.add("filtered");
  setTimeout(() => {
    container.classList.remove("filtered");
  }, 500);
}

/**
 * Show a modal with attempt details
 * @param {string} attemptId - The ID of the attempt to show
 */
function showAttemptDetailsModal(attemptId) {
  // In a real implementation, you would fetch the attempt
details from the API
  // For now, we'll just show a placeholder modal
```

```
  const modal = document.createElement("div");
  modal.className = "modal attempt-details-modal";
  modal.innerHTML = `
    <div class="modal-content">
      <span class="close-btn">&times;</span>
      <h2>Student Attempt Details</h2>
      <p>Detailed view for attempt ${attemptId} would be
shown here.</p>
      <p>It include:</p>
      <ul>
        <li>It Contain student name</li>
        <li>Time spent during exam</li>
        <li>Exam Start date and Time</li>
        <li>It also contain your score</li>
      </ul>
      <div class="modal-actions">
        <button class="close-modal-btn">Close</button>
      </div>
    </div>
  `;

  document.body.appendChild(modal);

  // Show the modal
  setTimeout(() => {
    modal.classList.add("visible");
  }, 10);

  // Set up event listeners
  const closeBtn = modal.querySelector(".close-btn");
  const closeModalBtn = modal.querySelector(".close-modal-
btn");

  closeBtn.addEventListener("click", () => {
    modal.classList.remove("visible");
    setTimeout(() => {
      document.body.removeChild(modal);
    }, 300);
```

```
  });

  closeModalBtn.addEventListener("click", () => {
    modal.classList.remove("visible");
    setTimeout(() => {
      document.body.removeChild(modal);
    }, 300);
  });

  modal.addEventListener("click", (e) => {
    if (e.target === modal) {
      modal.classList.remove("visible");
      setTimeout(() => {
        document.body.removeChild(modal);
      }, 300);
    }
  });
}

/**
 * Helper function to get percentage for chart bars
 * @param {number} value - The value
 * @param {number} total - The total
 * @returns {number} - The percentage (max 100)
 */
function getPercentage(value, total) {
  if (total === 0) return 0;
  return Math.min(Math.round((value / total) * 100), 100);
}

/**
 * Helper function to get time percentage
 * @param {number} timeSpent - Time spent in seconds
 * @param {number} timeLimit - Time limit in seconds
 * @returns {number} - The percentage (max 100)
 */
function getTimePercentage(timeSpent, timeLimit) {
  if (timeLimit === 0) return 0;
```

```
  return Math.min(Math.round((timeSpent / timeLimit) * 100),
100);
}

/**
 * Helper function to get success rate class
 * @param {number} rate - The success rate
 * @returns {string} - The CSS class
 */
function getSuccessRateClass(rate) {
  if (rate >= 80) return "excellent";
  if (rate >= 60) return "good";
  if (rate >= 40) return "average";
  return "poor";
}

/**
 * Helper function to get score class
 * @param {number} score - The score
 * @returns {string} - The CSS class
 */
function getScoreClass(score) {
  if (score >= 80) return "excellent";
  if (score >= 60) return "good";
  if (score >= 40) return "average";
  return "poor";
}

/**
 * Helper function to format duration in minutes
 * @param {number} minutes - Duration in minutes
 * @returns {string} - Formatted duration
 */
function formatDuration(minutes) {
  if (minutes < 1) return "Less than a minute";

  const hours = Math.floor(minutes / 60);
  const mins = Math.round(minutes % 60);
```

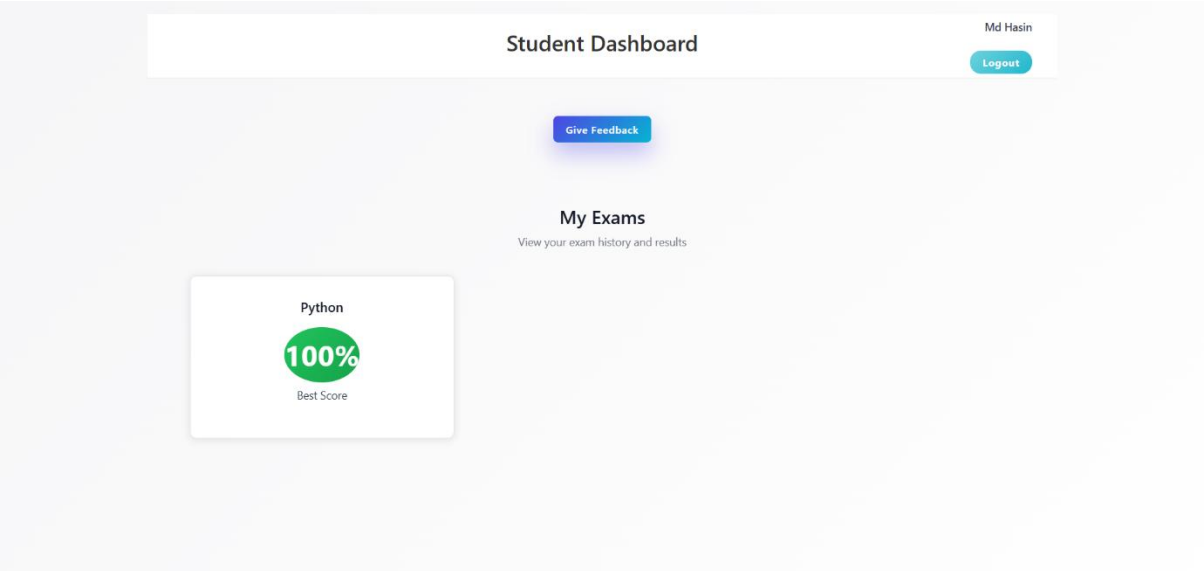
```
  if (hours === 0) {
    return `${mins} min${mins !== 1 ? "s" : ""}`;
  } else {
    return `${hours} hr${hours !== 1 ? "s" : ""} ${mins}
min${
      mins !== 1 ? "s" : ""
    }`;
  }
}

/**
 * Helper function to format seconds
 * @param {number} seconds - Time in seconds
 * @returns {string} - Formatted time
 */
function formatSeconds(seconds) {
  const mins = Math.floor(seconds / 60);
  const secs = Math.round(seconds % 60);

  if (mins === 0) {
    return `${secs}s`;
  } else {
    return `${mins}m ${secs}s`;
  }
}

/**
 * Helper function to format date
 * @param {Date} date - The date to format
 * @returns {string} - Formatted date
 */
function formatDate(date) {
  return (
    date.toLocaleDateString() +
    " " +
    date.toLocaleTimeString([], { hour: "2-digit", minute:
"2-digit" })
```

```
  );
}

/**
 * Helper function to truncate text
 * @param {string} text - The text to truncate
 * @param {number} maxLength - Maximum length
 * @returns {string} - Truncated text
 */
function truncateText(text, maxLength) {
  if (!text) return "";
  if (text.length <= maxLength) return text;
  return text.substring(0, maxLength) + "...";
}

/**
 * Helper function to parse time string
 * @param {string} timeString - Time string like "2m 30s"
 * @returns {number} - Time in seconds
 */
function parseTimeString(timeString) {
  let seconds = 0;

  // Match minutes
  const minutesMatch = timeString.match(/(\d+)m/);
  if (minutesMatch) {
    seconds += parseInt(minutesMatch[1]) * 60;
  }

  // Match seconds
  const secondsMatch = timeString.match(/(\d+)s/);
  if (secondsMatch) {
    seconds += parseInt(secondsMatch[1]);
  }

  return seconds;
}
```

```
function setupPdfDownload() {
  const btn = document.getElementById("downloadPdf");
  if (!btn) return;

  btn.addEventListener("click", async () => {
    const element =
document.getElementById("reportContent");

    const canvas = await html2canvas(element, {
      scale: 2,
      useCORS: true,
    });

    const imgData = canvas.toDataURL("image/png");

    const pdf = new jsPDF("p", "mm", "a4");

    const imgWidth = 210;
    const imgHeight = (canvas.height * imgWidth) /
canvas.width;

    let heightLeft = imgHeight;
    let position = 0;

    pdf.addImage(imgData, "PNG", 0, position, imgWidth,
imgHeight);
    heightLeft -= 295;

    while (heightLeft > 0) {
      position = heightLeft - imgHeight;
      pdf.addPage();
      pdf.addImage(imgData, "PNG", 0, position, imgWidth,
imgHeight);
      heightLeft -= 295;
    }
```

```
    pdf.save("Exam_Report.pdf");
  });
}
```

Snapshot of manage Student Dashboard:



StudentDashboard.js

```
import { fetchExamById } from "../api/exams.js";
import { fetchQuestionsByExamId } from
"../api/questions.js";
import { submitExamAttempt } from "../api/attempts.js";
import "../styles/student-exam.css";

/**
 * Renders the student exam view
 * @param {string} examId - The ID of the exam to take
 */
export async function renderStudentExam(examId) {
  const app = document.getElementById("app");
  app.innerHTML = `
    <div class="student-exam">
      <div class="loading">Loading exam details...</div>
    </div>
```

```
  `;

  try {
    // Check if user is authenticated
    const token = localStorage.getItem("token");
    if (!token) {
      // Redirect to login page with return URL
      const currentPath = window.location.pathname;
      history.pushState(
        null,
        "",
        `/?redirect=${encodeURIComponent(currentPath)}`
      );
      window.dispatchEvent(new Event("popstate"));
      return;
    }

    // Fetch exam details and questions in parallel
    const [examResponse, questionsResponse] = await
Promise.all([
      fetchExamById(examId),
      fetchQuestionsByExamId(examId),
    ]);

    // Extract the exam object from the response
    const exam = examResponse.exam || examResponse;

    // Store questions from the separate API call
    exam.questions = Array.isArray(questionsResponse) ?
questionsResponse : [];

    // Render the exam intro page
    renderExamIntro(app, exam);
  } catch (error) {
    console.error("Error loading exam:", error);
    app.querySelector(".student-exam").innerHTML = `
      <div class="error-message">
        <h3>Error Loading Exam</h3>
```

```
        <p>${
          error.message ||
          "Failed to load exam details. Please try again
later."
        }</p>
        <button id="back-btn" class="secondary-btn">Back to
Dashboard</button>
      </div>
    `;
    document.getElementById("back-
btn").addEventListener("click", () => {
      history.pushState(null, "", "/student/exams");
      window.dispatchEvent(new Event("popstate"));
    });
  }
}

/**
 * Renders the exam introduction page
 * @param {HTMLElement} app - The app container element
 * @param {Object} exam - The exam object with questions
 */
function renderExamIntro(app, exam) {
  app.innerHTML = `
    <div class="student-exam">
      <header class="exam-header">
        <h1>${exam.title}</h1>
        <div class="exam-meta">
          <span class="exam-audience">Target:
${exam.targetAudience}</span>
          <span class="exam-duration">Duration: ${
            exam.durationMinutes
          } minutes</span>
          <span class="exam-questions">Questions: ${
            exam.questions.length
          }</span>
          <span class="exam-points">Total Points:
${calculateTotalPoints(
```

```
            exam.questions
          )}</span>
        </div>
        <p class="exam-description">${
          exam.description || "No description provided"
        }</p>
      </header>

      <div class="exam-instructions">
        <h2>Instructions</h2>
        <ul>
          <li>This exam contains ${
            exam.questions.length
          } questions and has a time limit of ${
    exam.durationMinutes
  } minutes.</li>
          <li>Once you start the exam, the timer will begin
and cannot be paused.</li>
          <li>Each question has its own time limit. If you
don't answer within the time limit, the question will be
marked as incorrect.</li>
          <li>You must enable location access when prompted
to proceed with the exam.</li>
          <li>Do not refresh the page or navigate away
during the exam, or your progress must lost.</li>
          <li>Click the "Start Exam" button when you are
ready to begin.</li>
        </ul>
      </div>

      <div class="exam-actions">
        <button id="start-exam-btn" class="primary-
btn">Start Exam</button>
      </div>
    </div>
  `;

  // Set up event listener for start button
```

```
  document.getElementById("start-exam-
btn").addEventListener("click", () => {
    requestGeolocation(exam);
  });
}

/**
 * Request geolocation permission before starting the exam
 * @param {Object} exam - The exam object
 */
function requestGeolocation(exam) {
  const app = document.getElementById("app");

  app.innerHTML = `
    <div class="student-exam">
      <div class="geolocation-request">
        <h2>Location Access Required</h2>
        <p>This exam requires access to your location for
verification purposes.</p>
        <p>Please click "Allow" when prompted by your
browser.</p>
        <div class="loading-spinner"></div>
      </div>
    </div>
  `;

  if (navigator.geolocation) {
    navigator.geolocation.getCurrentPosition(
      (position) => {
        // Success - store coordinates
        const coordinates = {
          latitude: position.coords.latitude,
          longitude: position.coords.longitude,
        };

        // Start the exam with the coordinates
        startExam(exam, coordinates);
      },
```

```
      (error) => {
        // Error - show message
        app.innerHTML = `
          <div class="student-exam">
            <div class="error-message">
              <h3>Location Access Denied</h3>
              <p>You must allow location access to take this
exam. Please refresh the page and try again.</p>
              <p>Error: ${error.message}</p>
              <button id="retry-btn" class="primary-
btn">Retry</button>
            </div>
          </div>
        `;

        document.getElementById("retry-
btn").addEventListener("click", () => {
          requestGeolocation(exam);
        });
      }
    );
  } else {
    // Geolocation not supported
    app.innerHTML = `
      <div class="student-exam">
        <div class="error-message">
          <h3>Geolocation Not Supported</h3>
          <p>Your browser does not support geolocation,
which is required for this exam.</p>
          <p>Please try using a different browser or
device.</p>
          <button id="back-btn" class="secondary-
btn">Back</button>
        </div>
      </div>
    `;
```

```
    document.getElementById("back-
btn").addEventListener("click", () => {
      renderExamIntro(app, exam);
    });
  }
}

/**
 * Start the exam and show the first question
 * @param {Object} exam - The exam object
 * @param {Object} coordinates - The user's geolocation
coordinates
 */
function startExam(exam, coordinates) {
  // Initialize exam state
  const examState = {
    examId: exam._id || exam.id,
    startTime: new Date(),
    endTime: null,
    coordinates: coordinates,
    currentQuestionIndex: 0,
    answers: [],
    timeRemaining: exam.durationMinutes * 60, // in seconds
    examTimer: null,
  };

  // Start the exam timer
  examState.examTimer = setInterval(() => {
    examState.timeRemaining--;
    updateExamTimer(examState.timeRemaining);

    if (examState.timeRemaining <= 0) {
      // Time's up - end the exam
      clearInterval(examState.examTimer);
      finishExam(exam, examState);
    }
  }, 1000);
```

```
  // Show the first question
  showQuestion(exam, examState);
}

/**
 * Show a question to the student
 * @param {Object} exam - The exam object
 * @param {Object} examState - The current exam state
 */
function showQuestion(exam, examState) {
  const app = document.getElementById("app");
  const question =
exam.questions[examState.currentQuestionIndex];

  if (!question) {
    // No more questions - finish the exam
    finishExam(exam, examState);
    return;
  }

  // Initialize question timer
  let questionTimeRemaining = question.timeLimit;
  let questionTimer;

  // Prepare the question HTML based on type
  let questionContentHTML = "";

  if (question.type === "direct") {
    questionContentHTML = `
      <div class="question-input">
        <input type="text" id="direct-answer"
placeholder="Enter your answer" autocomplete="off">
      </div>
    `;
  } else if (question.type === "mcq") {
    questionContentHTML = `
      <div class="question-options">
        ${question.options
```

```
          .map(
            (option, index) => `
          <div class="option-item">
            <input type="radio" name="mcq-option"
id="option-${index}" value="${index}">
            <label for="option-${index}">${option}</label>
          </div>
        `
          )
          .join("")}
      </div>
    `;
  }

  // Render the question
  app.innerHTML = `
    <div class="student-exam question-view">
      <header class="exam-header">
        <div class="exam-progress">
          <div class="progress-bar">
            <div class="progress-fill" style="width: ${
              (examState.currentQuestionIndex /
exam.questions.length) * 100
            }%"></div>
          </div>
          <span class="progress-text">Question ${
            examState.currentQuestionIndex + 1
          } of ${exam.questions.length}</span>
        </div>
        <div class="timers">
          <div class="exam-timer">
            <span>Exam Time: </span>
            <span id="exam-time-remaining">${formatTime(
              examState.timeRemaining
            )}</span>
          </div>
          <div class="question-timer">
            <span>Question Time: </span>
```

```
            <span id="question-time-remaining">${formatTime(
              questionTimeRemaining
            )}</span>
          </div>
        </div>
      </header>

      <main class="question-container">
        <div class="question-header">
          <span class="question-points">${question.points}
points</span>
          <span class="question-type">${
            question.type === "direct" ? "Direct Question" :
"Multiple Choice"
          }</span>
        </div>

        <div class="question-text">
          <p>${question.text}</p>
        </div>

        ${question.attachment ?
renderAttachment(question.attachment) : ""}

        <div class="question-form">
          ${questionContentHTML}
        </div>

        <div class="question-actions">
          <button id="submit-answer-btn" class="primary-
btn">Submit Answer</button>
        </div>
      </main>
    </div>
  `;

  // Start question timer
  questionTimer = setInterval(() => {
```

```
    questionTimeRemaining--;
    document.getElementById("question-time-
remaining").textContent = formatTime(
      questionTimeRemaining
    );

    if (questionTimeRemaining <= 0) {
      // Time's up for this question
      clearInterval(questionTimer);

      // Record a blank answer (time expired)
      recordAnswer(exam, examState, question, null, true);

      // Move to next question
      examState.currentQuestionIndex++;
      showQuestion(exam, examState);
    }
  }, 1000);

  // Set up event listener for submit button
  document.getElementById("submit-answer-
btn").addEventListener("click", () => {
    // Stop the question timer
    clearInterval(questionTimer);

    // Get the answer based on question type
    let answer = null;

    if (question.type === "direct") {
      answer = document.getElementById("direct-
answer").value.trim();
    } else if (question.type === "mcq") {
      const selectedOption = document.querySelector(
        'input[name="mcq-option"]:checked'
      );
      if (selectedOption) {
        answer = parseInt(selectedOption.value);
      }
```

```
    }

    // Record the answer
    recordAnswer(exam, examState, question, answer, false);

    // Move to next question
    examState.currentQuestionIndex++;
    showQuestion(exam, examState);
  });
}

/**
 * Record an answer in the exam state
 * @param {Object} exam - The exam object
 * @param {Object} examState - The current exam state
 * @param {Object} question - The current question
 * @param {*} answer - The student's answer
 * @param {boolean} timeExpired - Whether the time expired
before answering
 */
function recordAnswer(exam, examState, question, answer,
timeExpired) {
  examState.answers.push({
    questionId: question._id,
    answer: answer,
    timeExpired: timeExpired,
    timestamp: new Date(),
  });
}

/**
 * Finish the exam and calculate the score
 * @param {Object} exam - The exam object
 * @param {Object} examState - The current exam state
 */
function finishExam(exam, examState) {
  // Clear any remaining timers
  if (examState.examTimer) {
```

```
    clearInterval(examState.examTimer);
  }

  // Set the end time
  examState.endTime = new Date();

  // Calculate the score
  const score = calculateScore(exam.questions,
examState.answers);
  const percentage = Math.round(
    (score.totalScore / score.maxPossibleScore) * 100
  );

  const app = document.getElementById("app");

  // Show the results
  app.innerHTML = `
    <div class="student-exam results-view">
      <header class="exam-header">
        <h1>Exam Completed</h1>
      </header>

      <main class="results-container">
        <div class="score-display">
          <div class="score-circle
${getScoreClass(percentage)}">
            <span class="score-
percentage">${percentage}%</span>
          </div>
          <div class="score-details">
            <p>You scored ${score.totalScore} out of ${
    score.maxPossibleScore
  } points</p>
            <p>Time taken: ${formatDuration(
              examState.startTime,
              examState.endTime
            )}</p>
          </div>
```

```
        </div>

        <div class="results-summary">
          <h2>Summary</h2>
          <div class="summary-stats">
            <div class="stat-item">
              <span class="stat-label">Total
Questions:</span>
              <span class="stat-
value">${exam.questions.length}</span>
            </div>
            <div class="stat-item">
              <span class="stat-label">Answered:</span>
              <span class="stat-
value">${examState.answers.length}</span>
            </div>
            <div class="stat-item">
              <span class="stat-label">Correct:</span>
              <span class="stat-
value">${score.correctAnswers}</span>
            </div>
            <div class="stat-item">
              <span class="stat-label">Incorrect:</span>
              <span class="stat-
value">${score.incorrectAnswers}</span>
            </div>
          </div>
        </div>

        <div class="results-actions">
          <button id="back-to-dashboard-btn" class="primary-
btn">Back to Dashboard</button>
        </div>
      </main>
    </div>
  `;

  // Submit the exam attempt to the server
```

```
  submitExamAttempt({
    examId: exam.id,
    startTime: examState.startTime,
    endTime: examState.endTime,
    coordinates: examState.coordinates,
    answers: examState.answers,
    score: percentage,
  }).catch((error) => {
    console.error("Error submitting exam attempt:", error);
    showNotification(
      "Your results have been saved locally, but there was
an error submitting them to the server.",
      "error"
    );
  });

  // Set up event listener for back button
  document
    .getElementById("back-to-dashboard-btn")
    .addEventListener("click", () => {
      history.pushState(null, "", "/student/exams");
      window.dispatchEvent(new Event("popstate"));
    });
}

/**
 * Calculate the score for the exam
 * @param {Array} questions - The exam questions
 * @param {Array} answers - The student's answers
 * @returns {Object} - Score details
 */
function calculateScore(questions, answers) {
  let totalScore = 0;
  let maxPossibleScore = 0;
  let correctAnswers = 0;
  let incorrectAnswers = 0;

  // Create a map of question IDs to answers for easy lookup
```

```
  const answerMap = {};
  answers.forEach((answer) => {
    answerMap[answer.questionId] = answer;
  });

  // Calculate score for each question
  questions.forEach((question) => {
    const points = question.points || 1;
    maxPossibleScore += points;

    const studentAnswer = answerMap[question._id];

    // If no answer or time expired, it's incorrect
    if (!studentAnswer || studentAnswer.timeExpired) {
      incorrectAnswers++;
      return;
    }

    let isCorrect = false;

    if (question.type === "direct") {
      // For direct questions, check if the answer matches
within tolerance
      const studentValue =
studentAnswer.answer.toLowerCase();
      const correctValue =
question.correctAnswer.toLowerCase();

      if (isNumeric(studentValue) &&
isNumeric(correctValue)) {
        // Numeric comparison with tolerance
        const studentNum = parseFloat(studentValue);
        const correctNum = parseFloat(correctValue);
        const tolerance = question.tolerance || 0;

        const percentDiff =
          Math.abs((studentNum - correctNum) / correctNum) *
100;
```

```
        isCorrect = percentDiff <= tolerance;
      } else {
        // String comparison
        isCorrect = studentValue === correctValue;
      }
    } else if (question.type === "mcq") {
      // For MCQ, check if the selected option is in the
correct options
      isCorrect =
question.correctOptions.includes(studentAnswer.answer);
    }

    if (isCorrect) {
      totalScore += points;
      correctAnswers++;
    } else {
      incorrectAnswers++;
    }
  });

  return {
    totalScore,
    maxPossibleScore,
    correctAnswers,
    incorrectAnswers,
  };
}

/**
 * Check if a string is a numeric value
 * @param {string} str - The string to check
 * @returns {boolean} - Whether the string is numeric
 */
function isNumeric(str) {
  return !isNaN(str) && !isNaN(parseFloat(str));
}

/**
```

```
 * Get the CSS class for a score percentage
 * @param {number} percentage - The score percentage
 * @returns {string} - The CSS class
 */
function getScoreClass(percentage) {
  if (percentage >= 80) return "excellent";
  if (percentage >= 60) return "good";
  if (percentage >= 40) return "average";
  return "poor";
}

/**
 * Format a time duration between two dates
 * @param {Date} startTime - The start time
 * @param {Date} endTime - The end time
 * @returns {string} - Formatted duration
 */
function formatDuration(startTime, endTime) {
  const durationMs = endTime - startTime;
  const seconds = Math.floor(durationMs / 1000) % 60;
  const minutes = Math.floor(durationMs / (1000 * 60)) % 60;
  const hours = Math.floor(durationMs / (1000 * 60 * 60));

  if (hours > 0) {
    return `${hours}h ${minutes}m ${seconds}s`;
  } else {
    return `${minutes}m ${seconds}s`;
  }
}

/**
 * Update the exam timer display
 * @param {number} timeRemaining - Time remaining in seconds
 */
function updateExamTimer(timeRemaining) {
  const timerElement = document.getElementById("exam-time-
remaining");
  if (timerElement) {
```

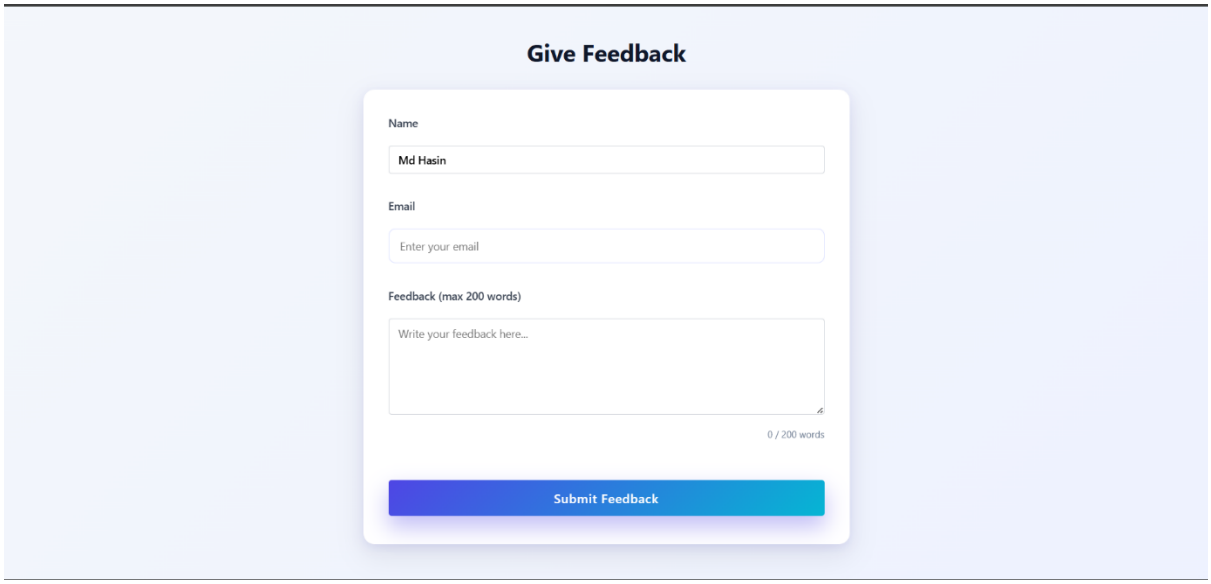
```
    timerElement.textContent = formatTime(timeRemaining);

    // Add warning class when time is running low
    if (timeRemaining <= 300) {
      // 5 minutes
      timerElement.classList.add("time-warning");
    }
    if (timeRemaining <= 60) {
      // 1 minute
      timerElement.classList.add("time-critical");
    }
  }
}

/**
 * Format time in seconds to MM:SS format
 * @param {number} seconds - Time in seconds
 * @returns {string} - Formatted time
 */
function formatTime(seconds) {
  const mins = Math.floor(seconds / 60);
  const secs = seconds % 60;
  return `${mins.toString().padStart(2, "0")}:${secs
    .toString()
    .padStart(2, "0")}`;
}

/**
 * Calculate total points for the exam questions
 * @param {Array} questions - Array of question objects
 * @returns {number} - Total points
 */
function calculateTotalPoints(questions) {
  if (!questions || !Array.isArray(questions)) return 0;
  return questions.reduce(
    (total, question) => total + (question.points || 1),
    0
  );
```

```
}

/**
 * Renders an attachment based on its type
 * @param {Object} attachment - The attachment object
 * @returns {string} - HTML string of the attachment
 */
function renderAttachment(attachment) {
  if (!attachment) return "";

  switch (attachment.type) {
    case "image":
      return `
        <div class="attachment image-attachment">
          <img src="${attachment.url}" alt="Question
attachment" class="question-image">
        </div>
      `;
    case "audio":
      return `
        <div class="attachment audio-attachment">
          <audio controls>
            <source src="${attachment.url}"
type="audio/mpeg">
            Your browser does not support audio
          </audio>
        </div>
      `;
    case "video":
      return `
        <div class="attachment video-attachment">
          <video controls>
            <source src="${attachment.url}"
type="video/mp4">
            Your browser does not support video
          </video>
        </div>
      `;
```

```
    default:
      return "";
  }
}

/**
 * Shows a notification message
 * @param {string} message - The message to display
 * @param {string} type - The type of notification (success,
error, info)
 */
function showNotification(message, type = "info") {
  // Check if notification container exists, create if not
  let notificationContainer =
document.querySelector(".notification-container");
  if (!notificationContainer) {
    notificationContainer = document.createElement("div");
    notificationContainer.className = "notification-
container";
    document.body.appendChild(notificationContainer);
  }

  // Create notification element
  const notification = document.createElement("div");
  notification.className = `notification ${type}`;
  notification.innerHTML = `
    <div class="notification-content">
      <span class="notification-message">${message}</span>
      <button class="notification-close">&times;</button>
    </div>
  `;

  // Add to container
  notificationContainer.appendChild(notification);

  // Add close button functionality
  notification
    .querySelector(".notification-close")
```

```
    .addEventListener("click", () => {
      notification.classList.add("fade-out");
      setTimeout(() => {
        notificationContainer.removeChild(notification);
      }, 300);
    });

  // Auto remove after 5 seconds
  setTimeout(() => {
    if (notification.parentNode) {
      notification.classList.add("fade-out");
      setTimeout(() => {
        if (notification.parentNode) {
          notificationContainer.removeChild(notification);
        }
      }, 300);
    }
  }, 5000);
}
```

Snapshot of Give Feedback:



GiveFeedback.js

```
import { getLoggedInUser } from "../api/auth.js";
import "../styles/student-feedback.css";


/**
 * ================================
 * RENDER GIVE FEEDBACK PAGE
 * ================================
 */
export function renderGiveFeedback() {
  const app = document.getElementById("app");
  const user = getLoggedInUser();

  // 🔐 STUDENT ONLY ACCESS
  if (!user || user.userType !== "student") {
    history.pushState(null, "", "/");
    window.dispatchEvent(new Event("popstate"));
    return;
  }

  // ================================
  // UI
  // ================================
  app.innerHTML = `
    <section class="feedback-page">
      <h1>Give Feedback</h1>

      <form id="feedback-form" class="feedback-form">

        <!-- NAME -->
        <div class="form-group">
          <label>Name</label>
          <input
            type="text"
            id="name"
            value="${user.name || ""}"
```

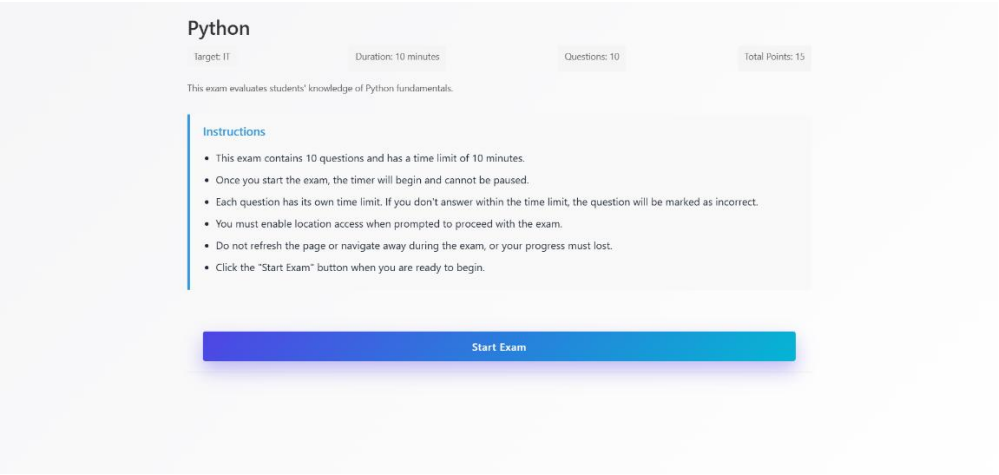
```
          readonly
        />
      </div>

      <!-- EMAIL -->
      <div class="form-group">
        <label>Email</label>
        <input
          type="email"
          id="email"
          placeholder="Enter your email"
          value="${user.email || ""}"
          required
        />
      </div>

      <!-- MESSAGE -->
      <div class="form-group">
        <label>Feedback (max 200 words)</label>
        <textarea
          id="message"
          placeholder="Write your feedback here..."
          maxlength="200"
          rows="5"
          required
        ></textarea>
        <small id="word-count">0 / 200 words</small>
      </div>

      <!-- SUBMIT -->
      <button type="submit" class="primary-btn">
        Submit Feedback
      </button>

    </form>
  </section>
`;
```

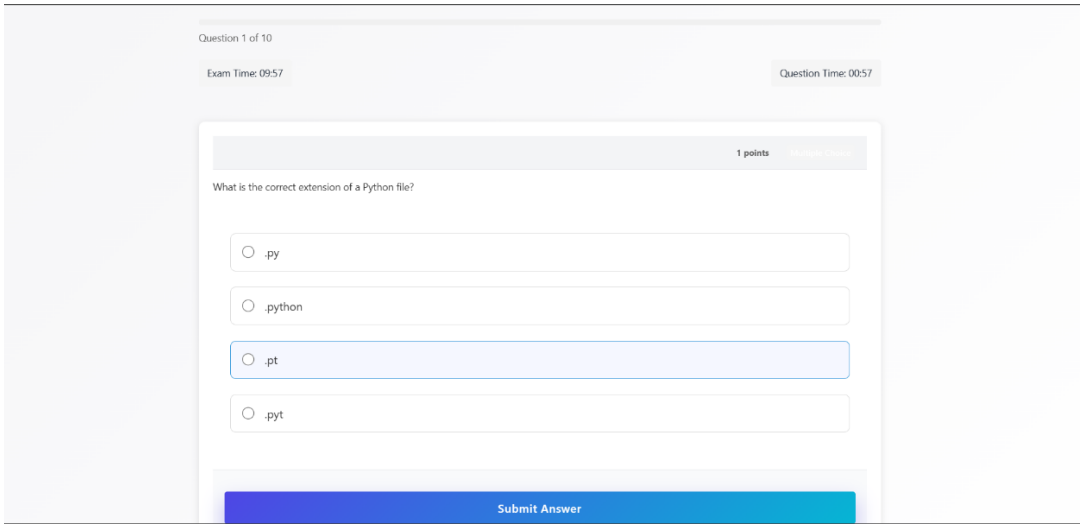
```
  // ================================
  // WORD COUNT LOGIC
  // ================================
  const messageInput = document.getElementById("message");
  const wordCountEl = document.getElementById("word-count");

  messageInput.addEventListener("input", () => {
    const words = messageInput.value.trim()
      ? messageInput.value.trim().split(/\s+/).length
      : 0;
    wordCountEl.textContent = `${words} / 200 words`;
  });

  // ================================
  // FORM SUBMIT
  // ================================
  document
    .getElementById("feedback-form")
    .addEventListener("submit", submitFeedback);
}

/**
 * ================================
 * SUBMIT FEEDBACK HANDLER
 * ================================
 */
async function submitFeedback(e) {
  e.preventDefault();

  const data = {
    name: document.getElementById("name").value.trim(),
    email: document.getElementById("email").value.trim(),
    message:
document.getElementById("message").value.trim(),
  };

  // ================================
  // CLIENT VALIDATION
```

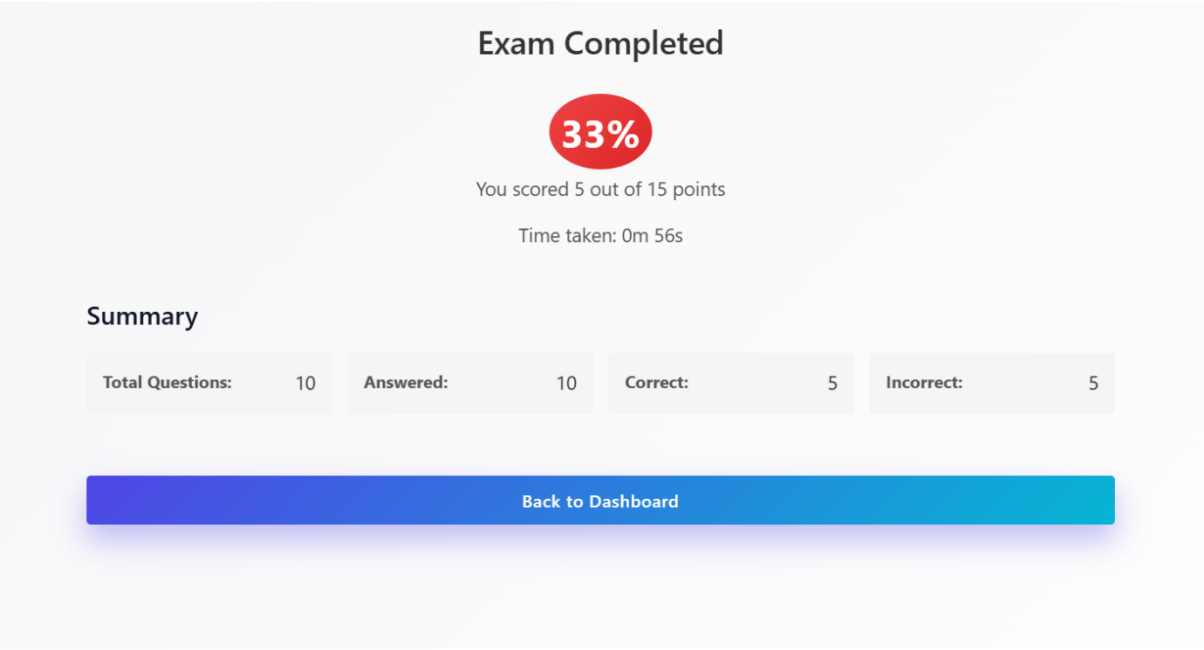
```
  // ================================
  if (!data.email) {
    alert("❌ Email is required");
    return;
  }

  if (!data.message) {
    alert("❌ Feedback message is required");
    return;
  }

  if (data.message.split(/\s+/).length > 200) {
    alert("❌ Feedback must be within 200 words");
    return;
  }

  // ================================
  // API CALL (🔥 FIXED URL)
  // ================================
  try {
    const res = await
fetch("http://localhost:3000/api/feedback", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        Authorization: `Bearer
${localStorage.getItem("token")}`,
      },
      body: JSON.stringify(data),
    });

    if (!res.ok) {
      const err = await res.json();
      throw new Error(err.message || "Submit failed");
    }

    // ================================
    // SUCCESS
```

```
    // ================================
    alert("✅ Feedback submitted successfully");

    history.pushState(null, "", "/student/exams");
    window.dispatchEvent(new Event("popstate"));

  } catch (err) {
    console.error("Feedback submit error:", err);
    alert("❌ Failed to submit feedback");
  }
}
```

Snapshot of Exam interface and result generation:







ExamInterface.js:

```
import { fetchExamById, deleteExam, updateExam } from
"../api/exams.js";
import {
  createQuestion,
  fetchQuestionsByExamId,
  deleteQuestion,
  updateQuestion,
  uploadQuestionAttachment,
  fetchQuestionById,
} from "../api/questions.js";

import { getLoggedInUser } from "../api/auth.js"; // ✅ YEH
LINE ADD KARO

import "../styles/exam-interface.css";


/**
 * Renders the exam editor interface
 * @param {string} examId - The ID of the exam to edit
 */
```

```
export async function renderExamEditor(examId) {
  const app = document.getElementById("app");

  // 🔐 ROLE + LOGIN CHECK (YAHI SABSE IMPORTANT HAI)
  const user = getLoggedInUser();

  if (!user || user.userType !== "teacher") {
    alert("❌ Unauthorized access");
    history.pushState(null, "", "/");
    window.dispatchEvent(new Event("popstate"));
    return;
  }

  // 👇 Iske baad hi UI aur API calls
  app.innerHTML = `
    <div class="exam-editor">
      <div class="loading">Loading exam details...</div>
    </div>
  `;

  try {
    // Store the original examId for later use
    const originalExamId = examId;

    const [examResponse, questionsResponse] = await
Promise.all([
      fetchExamById(examId),
      fetchQuestionsByExamId(examId),
    ]);


    // Extract the exam object from the response
    const exam = examResponse.exam || examResponse;

    // Store the original ID to use for API calls
    exam.originalId = originalExamId;

    // Ensure the exam has an ID property
    if (!exam._id && !exam.examId) {
```

```
      exam.examId = examId;
    }

    // Store questions from the separate API call
    exam.questions = Array.isArray(questionsResponse) ?
questionsResponse : [];

    renderExamDetails(app, exam);
  } catch (error) {
    console.error("Error loading exam:", error);
    app.querySelector(".exam-editor").innerHTML = `
      <div class="error-message">
        <h3>Error Loading Exam</h3>
        <p>${
          error.message ||
          "Failed to load exam details. Please try again
later."
        }</p>
        <button id="back-btn" class="secondary-btn">Back to
Dashboard</button>
      </div>
    `;
    document.getElementById("back-
btn").addEventListener("click", () => {
      history.pushState(null, "", "/teacher/exams");
      window.dispatchEvent(new Event("popstate"));
    });
  }
}

/**
 * Renders the exam details and questions
 * @param {HTMLElement} app - The app container element
 * @param {Object} exam - The exam object with questions
 */
function renderExamDetails(app, exam) {
  // Make sure we have the examId available - use the
original ID that worked with the API
```

```
  const examId = exam.originalId || exam._id || exam.examId
|| exam.id;

  app.innerHTML = `
    <div class="exam-editor">
      <header class="exam-header">
        <div class="breadcrumb">
          <a href="/teacher/exams" id="back-to-dashboard"
data-link="spa">Dashboard</a> / <span>Exam Editor</span>
        </div>
        <div class="exam-title-section">
          <h1>${exam.title}</h1>
          <span class="exam-status
${exam.status}">${exam.status}</span>
        </div>
        <div class="exam-meta">
          <span class="exam-audience">Target:
${exam.targetAudience}</span>
          <span class="exam-date">Created: ${new Date(
            exam.createdAt
          ).toLocaleDateString()}</span>
          ${
            exam.scheduledAt
              ? `<span class="exam-scheduled">Scheduled:
${new Date(
                  exam.scheduledAt
                ).toLocaleString()}</span>`
              : ""
          }
        </div>
        <p class="exam-description">${
          exam.description || "No description provided"
        }</p>



        <div class="exam-actions-header">
```

```
          <button id="publish-exam-btn" class="primary-btn
${
            exam.status === "published" ? "published" : ""
          }">
            ${exam.status === "published" ? "Published ✓" :
"Publish Exam"}
          </button>
          ${
            exam.status === "published"
              ? `<button id="view-statistics-btn"
class="stats-btn">View Statistics</button>`
              : ""
          }
          <button id="delete-exam-btn" class="danger-
btn">Delete Exam</button>
        </div>

        ${
          exam.status === "published"
            ? `
          <div class="exam-share-link">
            <p>Share this link with your students:</p>
            <div class="link-container">
              <input type="text" readonly value="${
                window.location.origin
              }/exam/${exam.id || exam._id}" id="share-
link">
              <button id="copy-link-btn" class="secondary-
btn">Copy</button>
            </div>
          </div>
        `
            : ""
        }
      </header>

      <main class="exam-content">
        <div class="exam-actions">
```

```
          <button id="add-question-btn" class="primary-
btn">Add Question</button>
          <div class="exam-stats">
            <span class="question-count">${
              exam.questions.length
            } questions</span>
            <span class="total-
points">${calculateTotalPoints(
              exam.questions
            )} total points</span>
            ${
              exam.durationMinutes
                ? `<span
class="duration">${exam.durationMinutes} minutes</span>`
                : ""
            }
          </div>
        </div>

        <div id="questions-container" class="questions-
container">
          ${renderQuestionsList(exam.questions)}
        </div>
      </main>
    </div>
  `;

  // Set up event listeners
  setupEventListeners(exam);
}

/**
 * Calculate total points for the exam questions
 * @param {Array} questions - Array of question objects
 * @returns {number} - Total points
 */
function calculateTotalPoints(questions) {
  if (!questions || !Array.isArray(questions)) return 0;
```

```
  return questions.reduce(
    (total, question) => total + (question.points || 1),
    0
  );
}

/**
 * Renders the list of questions
 * @param {Array} questions - Array of question objects
 * @returns {string} - HTML string of question cards
 */
function renderQuestionsList(questions) {
  if (!questions || !Array.isArray(questions) ||
questions.length === 0) {
    return `
      <div class="empty-state">
        <div class="empty-icon">📝</div>
        <h3>No questions added yet</h3>
        <p>Click the "Add Question" button to create your
first question.</p>
      </div>
    `;
  }

  return questions
    .map(
      (question, index) => `
    <div class="question-card" data-id="${question._id}">
      <div class="question-header">
        <span class="question-number">Q${index + 1}</span>
        <span class="question-type ${question.type}">${
        question.type === "direct" ? "Direct" : "MCQ"
      }</span>
        <span class="question-points">${question.points}
pts</span>
        <span class="question-
time">${question.timeLimit}s</span>
      </div>
```

```
      <div class="question-content">
        <p>${question.text}</p>
        ${question.attachment ?
renderAttachment(question.attachment) : ""}

        ${renderQuestionDetails(question)}
      </div>
      <div class="question-actions">
        <button class="edit-btn" data-
id="${question._id}">Edit</button>
        <button class="delete-btn" data-
id="${question._id}">Delete</button>
      </div>
    </div>
  `
    )
    .join("");
}

/**
 * Renders question-specific details based on type
 * @param {Object} question - The question object
 * @returns {string} - HTML string of question details
 */
function renderQuestionDetails(question) {
  if (question.type === "direct") {
    return `
      <div class="question-details">
        <div class="answer-preview">
          <strong>Correct Answer:</strong>
${question.correctAnswer}
          ${
            question.tolerance
              ? `<span>(Tolerance:
${question.tolerance}%)</span>`
              : ""
          }
        </div>
```

```
      </div>
    `;
  } else if (question.type === "mcq") {
    return `
      <div class="question-details">
        <div class="options-list">
          ${question.options
            .map(
              (option, index) => `
            <div class="option ${
              question.correctOptions.includes(index) ?
"correct" : ""
            }">
              <span class="option-
marker">${String.fromCharCode(
                65 + index
              )}</span>
              <span class="option-text">${option}</span>
              ${
                question.correctOptions.includes(index)
                  ? '<span class="correct-marker">✓</span>'
                  : ""
              }
            </div>
          `
            )
            .join("")}
        </div>
      </div>
    `;
  }
  return "";
}

/**
 * Renders an attachment based on its type
 * @param {Object} attachment - The attachment object
 * @returns {string} - HTML string of the attachment
```

```
 */
function renderAttachment(attachment) {
  if (!attachment) return "";

  switch (attachment.type) {
    case "image":
      return `
        <div class="attachment image-attachment">
          <img src="${attachment.url}" alt="Question
attachment" class="question-image">
        </div>
      `;
    case "audio":
      return `
        <div class="attachment audio-attachment">
          <audio controls>
            <source src="${attachment.url}"
type="audio/mpeg">
            Your browser does not support audio
          </audio>
        </div>
      `;
    case "video":
      return `
        <div class="attachment video-attachment">
          <video controls>
            <source src="${attachment.url}"
type="video/mp4">
            Your browser does not support video
          </video>
        </div>
      `;
    default:
      return "";
  }
}

/**
```

```
 * Sets up all event listeners for the exam interface
 * @param {Object} exam - The exam object
 */
function setupEventListeners(exam) {
  // Store the exam ID that works with the API
  const examId = exam.originalId || exam._id || exam.examId
|| exam.id;

  // Back to dashboard
  document
    .getElementById("back-to-dashboard")
    .addEventListener("click", (e) => {
      e.preventDefault();
      history.pushState(null, "", "/teacher/exams");
      window.dispatchEvent(new Event("popstate"));
    });

  // View Statistics button (only for published exams)
  const viewStatsBtn = document.getElementById("view-
statistics-btn");
  if (viewStatsBtn) {
    viewStatsBtn.addEventListener("click", () => {
      history.pushState(null, "",
`/teacher/exams/${examId}/statistics`);
      window.dispatchEvent(new Event("popstate"));
    });
  }

  // Delete Exam button
  document.getElementById("delete-exam-
btn").addEventListener("click", () => {
    showDeleteExamDialog(examId);
  });

  // Publish Exam button
  document
    .getElementById("publish-exam-btn")
    .addEventListener("click", async () => {
```

```
      if (exam.status === "published") {
        // Already published, show the share link
        const shareLink = document.querySelector(".exam-
share-link");
        if (shareLink) {
          shareLink.classList.add("highlight");
          setTimeout(() => {
            shareLink.classList.remove("highlight");
          }, 1500);
        }
        return;
      }

      // Show confirmation dialog before publishing
      showPublishExamDialog(examId, exam);
    });

  // Copy link button
  const copyLinkBtn = document.getElementById("copy-link-
btn");
  if (copyLinkBtn) {
    copyLinkBtn.addEventListener("click", () => {
      const shareLink = document.getElementById("share-
link");
      shareLink.select();
      document.execCommand("copy");

      // Change button text temporarily
      const originalText = copyLinkBtn.textContent;
      copyLinkBtn.textContent = "Copied!";
      setTimeout(() => {
        copyLinkBtn.textContent = originalText;
      }, 2000);
    });
  }

  // Add question button
```

```
  const addQuestionBtn = document.getElementById("add-
question-btn");
  if (addQuestionBtn) {
    addQuestionBtn.addEventListener("click", () => {
      console.log("Add question button clicked");
      openQuestionDialog(null, examId);
    });
  } else {
    console.error("Add question button not found");
  }

  // Edit question buttons
  document.querySelectorAll(".edit-btn").forEach((btn) => {
    btn.addEventListener("click", async () => {
      const questionId = btn.dataset.id;
      try {
        // Fetch the latest question data
        const response = await
fetchQuestionById(questionId);
        const question = response.question || response;

        if (question) {
          openQuestionDialog(question, examId);
        } else {
          throw new Error("Question data not found");
        }
      } catch (error) {
        console.error("Error fetching question:", error);
        showNotification(
          "Failed to load question details: " +
error.message,
          "error"
        );
      }
    });
  });

  // Delete question buttons
```

```
  document.querySelectorAll(".delete-btn").forEach((btn) =>
{
    btn.addEventListener("click", async () => {
      // Create a custom confirmation dialog
      const confirmDialog = document.createElement("div");
      confirmDialog.className = "confirm-dialog-overlay";
      confirmDialog.innerHTML = `
        <div class="confirm-dialog">
          <h3>Delete Question</h3>
          <p>Are you sure you want to delete this question?
This action cannot be undone.</p>
          <div class="confirm-actions">
            <button class="cancel-delete secondary-
btn">Cancel</button>
            <button class="confirm-delete primary-
btn">Delete</button>
          </div>
        </div>
      `;
      document.body.appendChild(confirmDialog);

      // Add event listeners to the dialog buttons
      confirmDialog
        .querySelector(".cancel-delete")
        .addEventListener("click", () => {
          document.body.removeChild(confirmDialog);
        });

      confirmDialog
        .querySelector(".confirm-delete")
        .addEventListener("click", async () => {
          document.body.removeChild(confirmDialog);

          try {
            const questionId = btn.dataset.id;
            await deleteQuestion(questionId);
            showNotification("Question deleted
successfully", "success");
```

```
            // Use the stored exam ID that works with the
API
            await renderExamEditor(examId);
          } catch (error) {
            console.error("Error deleting question:",
error);
            showNotification(
              "Failed to delete question: " + error.message,
              "error"
            );
          }
        });
    });
  });

  // Modal close button
  // document.querySelector(".close-
btn").addEventListener("click", () => {
  //   closeQuestionDialog()
  // })

  // Close modal when clicking outside
  // document.getElementById("question-
dialog").addEventListener("click", (e) => {
  //   if (e.target === document.getElementById("question-
dialog")) {
  //     closeQuestionDialog()
  //   }
  // })
}

/**
 * Shows a dialog to confirm exam deletion
 * @param {string} examId - The ID of the exam to delete
 */
function showDeleteExamDialog(examId) {
  const confirmDialog = document.createElement("div");
```

```
  confirmDialog.className = "confirm-dialog-overlay";
  confirmDialog.innerHTML = `
    <div class="confirm-dialog delete-exam-dialog">
      <h3>Delete Exam</h3>
      <p>Are you sure you want to delete this entire exam
and all its questions? This action cannot be undone.</p>
      <div class="confirm-actions">
        <button class="cancel-delete secondary-
btn">Cancel</button>
        <button class="confirm-delete danger-btn">Delete
Exam</button>
      </div>
    </div>
  `;
  document.body.appendChild(confirmDialog);

  // Add event listeners to the dialog buttons
  confirmDialog
    .querySelector(".cancel-delete")
    .addEventListener("click", () => {
      document.body.removeChild(confirmDialog);
    });

  confirmDialog
    .querySelector(".confirm-delete")
    .addEventListener("click", async () => {
      document.body.removeChild(confirmDialog);

      try {
        // Show loading overlay
        const loadingOverlay =
document.createElement("div");
        loadingOverlay.className = "loading-overlay";
        loadingOverlay.innerHTML = `
        <div class="loading-spinner"></div>
        <p>Deleting exam...</p>
      `;
        // document.body.appendChild(loadingOverlay);
```

```
        // Delete the exam
        await deleteExam(examId);

        // Show success notification
        showNotification("Exam deleted successfully",
"success");

        // Redirect to dashboard
        history.pushState(null, "", "/teacher/exams");
        window.dispatchEvent(new Event("popstate"));
      } catch (error) {
        console.error("Error deleting exam:", error);
        showNotification("Failed to delete exam: " +
error.message, "error");

        // Remove loading overlay if it exists
        const loadingOverlay =
document.querySelector(".loading-overlay");
        if (loadingOverlay) {
          document.body.removeChild(loadingOverlay);
        }
      }
    });
}

/**
 * Shows a dialog to confirm exam publication
 * @param {string} examId - The ID of the exam to publish
 * @param {Object} exam - The exam object
 */
function showPublishExamDialog(examId, exam) {
  // Check if the exam has questions
  if (!exam.questions || exam.questions.length === 0) {
    showNotification(
      "Cannot publish an exam with no questions. Add at
least one question first.",
      "error"
```

```
    );
    return;
  }

  const confirmDialog = document.createElement("div");
  confirmDialog.className = "confirm-dialog-overlay";
  confirmDialog.innerHTML = `
    <div class="confirm-dialog publish-exam-dialog">
      <h3>Publish Exam</h3>
      <p>Are you ready to publish this exam? Once published,
students will be able to access it using the provided
link.</p>
      <div class="form-group">
        <label for="exam-duration">Exam Duration
(minutes)</label>
        <input type="number" id="exam-duration" min="5"
value="${
          exam.durationMinutes || 60
        }" required>
      </div>
      <div class="confirm-actions">
        <button class="cancel-publish secondary-
btn">Cancel</button>
        <button class="confirm-publish primary-btn">Publish
Exam</button>
      </div>
    </div>
  `;
  document.body.appendChild(confirmDialog);

  // Add event listeners to the dialog buttons
  confirmDialog
    .querySelector(".cancel-publish")
    .addEventListener("click", () => {
      document.body.removeChild(confirmDialog);
    });

  confirmDialog
```

```
    .querySelector(".confirm-publish")
    .addEventListener("click", async () => {
      const durationInput = document.getElementById("exam-
duration");
      const durationMinutes =
Number.parseInt(durationInput.value);

      if (!durationMinutes || durationMinutes < 5) {
        durationInput.setCustomValidity(
          "Please enter a valid duration (minimum 5
minutes)"
        );
        durationInput.reportValidity();
        return;
      }

      document.body.removeChild(confirmDialog);

      try {
        // Show loading overlay
        const loadingOverlay =
document.createElement("div");
        loadingOverlay.className = "loading-overlay";
        loadingOverlay.innerHTML = `
        <div class="loading-spinner"></div>
        <p>Publishing exam...</p>
      `;
        document.body.appendChild(loadingOverlay);

        // Update the exam status to published
        const updatedExam = {
          ...exam,
          status: "published",
          durationMinutes: durationMinutes,
        };

        await updateExam(examId, updatedExam);
```

```
        // Show success notification
        showNotification("Exam published successfully",
"success");

        // Refresh the exam view
        await renderExamEditor(examId);

        // Remove loading overlay
        document.body.removeChild(loadingOverlay);
      } catch (error) {
        console.error("Error publishing exam:", error);
        showNotification("Failed to publish exam: " +
error.message, "error");

        // Remove loading overlay if it exists
        const loadingOverlay =
document.querySelector(".loading-overlay");
        if (loadingOverlay) {
          document.body.removeChild(loadingOverlay);
        }
      }
    });
}

/**
 * Opens the question dialog for adding or editing a
question
 * @param {Object} question - The question object to edit
(null for new question)
 * @param {string} examId - The ID of the exam
 */
function openQuestionDialog(question = null, examId) {
  console.log("Opening question dialog with dynamic
creation");

  // Create the dialog overlay
  const dialogOverlay = document.createElement("div");
```

```
  dialogOverlay.className = "confirm-dialog-overlay
question-dialog-overlay";

  // Determine if we're adding or editing
  const isEditing = question !== null;
  const dialogTitle = isEditing ? "Edit Question" : "Add New
Question";

  // Create the dialog content
  dialogOverlay.innerHTML = `
    <div class="confirm-dialog question-dialog">
      <div class="dialog-header">
        <h3>${dialogTitle}</h3>
        <button class="close-dialog-btn">&times;</button>
      </div>

      <form id="question-form" class="question-form">
        <div class="form-group">
          <label for="question-type">Question Type</label>
          <select id="question-type" required>
            <option value="">Select type</option>
            <option value="direct" ${
              isEditing && question.type === "direct" ?
"selected" : ""
            }>Direct Question</option>
            <option value="mcq" ${
              isEditing && question.type === "mcq" ?
"selected" : ""
            }>Multiple Choice (MCQ)</option>
          </select>
        </div>

        <div class="form-group">
          <label for="question-text">Question Text</label>
          <textarea id="question-text" required>${
            isEditing ? question.text : ""
          }</textarea>
        </div>
```

```
        <div class="form-group">
          <label for="question-attachment">Attachment
(Optional)</label>
          <input type="file" id="question-attachment"
accept="image/*,audio/*,video/*">
          <div id="attachment-preview" class="attachment-
preview ${
            isEditing && question.attachment ? "" : "hidden"
          }">
            ${
              isEditing && question.attachment
                ? `
                <div class="attachment-info">
                  <span class="attachment-name">${
                    question.attachment.filename ||
"Attachment"
                  }</span>
                  <span class="attachment-type">(${
                    question.attachment.type
                  })</span>
                  <button type="button" class="remove-
attachment-btn">&times;</button>
                </div>
                <input type="hidden" id="attachment-data"
value='${JSON.stringify(
                  question.attachment
                )}'>
              `
                : ""
            }
          </div>
        </div>

        <div id="direct-question-fields" class="question-
type-fields ${
          isEditing && question.type === "direct" ? "" :
"hidden"
```

```
        }">
          <div class="form-group">
            <label for="correct-answer">Correct
Answer</label>
            <input type="text" id="correct-answer" ${
              isEditing && question.type === "direct"
                ? 'value="' + (question.correctAnswer || "")
+ '"'
                : ""
            }>
          </div>
          <div class="form-group">
            <label for="tolerance">Tolerance Percentage (0-
100)</label>
            <input type="number" id="tolerance" min="0"
max="100" value="${
              isEditing && question.type === "direct"
                ? question.tolerance || 10
                : 10
            }">
          </div>
        </div>

        <div id="mcq-fields" class="question-type-fields ${
          isEditing && question.type === "mcq" ? "" :
"hidden"
        }">
          <div class="form-group">
            <label>Options (Add at least 2)</label>
            <div id="options-container">
              ${
                isEditing &&
                question.type === "mcq" &&
                Array.isArray(question.options)
                  ? question.options
                      .map(
                        (option, index) => `
                      <div class="option-item">
```

```
                        <input type="text" class="option-
input" value="${option.replace(
                          /"/g,
                          "&quot;"
                        )}">
                        <input type="checkbox"
class="correct-option" ${
                          Array.isArray(question.correctOpti
ons) &&
                          question.correctOptions.includes(i
ndex)
                            ? "checked"
                            : ""
                        }>
                        <span>Correct</span>
                        <button type="button" class="remove-
option-btn">&times;</button>
                      </div>
                    `
                      )
                      .join("")
                  : `
                    <div class="option-item">
                      <input type="text" class="option-
input" placeholder="Option text">
                      <input type="checkbox" class="correct-
option">
                      <span>Correct</span>
                    </div>
                    <div class="option-item">
                      <input type="text" class="option-
input" placeholder="Option text">
                      <input type="checkbox" class="correct-
option">
                      <span>Correct</span>
                    </div>
                  `
              }
```

```
            </div>
            <button type="button" id="add-option-btn"
class="secondary-btn">Add Option</button>
          </div>
        </div>

        <div class="form-group">
          <label for="question-points">Points</label>
          <input type="number" id="question-points" min="1"
value="${
            isEditing ? question.points || 1 : 1
          }" required>
        </div>

        <div class="form-group">
          <label for="time-limit">Time Limit
(seconds)</label>
          <input type="number" id="time-limit" min="15"
value="${
            isEditing ? question.timeLimit || 60 : 60
          }" required>
        </div>

        <input type="hidden" id="question-id" value="${
          isEditing ? question._id : ""
        }">
        <input type="hidden" id="exam-id" value="${examId}">

        <div class="form-actions">
          <button type="button" id="cancel-btn"
class="secondary-btn">Cancel</button>
          <button type="submit" class="primary-btn">Save
Question</button>
        </div>
      </form>
    </div>
  `;
```

```
  // Add the dialog to the DOM
  document.body.appendChild(dialogOverlay);

  // Set up event listeners

  // Close button
  dialogOverlay
    .querySelector(".close-dialog-btn")
    .addEventListener("click", () => {
      document.body.removeChild(dialogOverlay);
    });

  // Cancel button
  dialogOverlay.querySelector("#cancel-
btn").addEventListener("click", () => {
    document.body.removeChild(dialogOverlay);
  });

  // Click outside to close
  dialogOverlay.addEventListener("click", (e) => {
    if (e.target === dialogOverlay) {
      document.body.removeChild(dialogOverlay);
    }
  });

  // Question type change
  dialogOverlay
    .querySelector("#question-type")
    .addEventListener("change", (e) => {
      const directFields = dialogOverlay.querySelector(
        "#direct-question-fields"
      );
      const mcqFields = dialogOverlay.querySelector("#mcq-
fields");

      directFields.classList.add("hidden");
      mcqFields.classList.add("hidden");
```

```
      if (e.target.value === "direct") {
        directFields.classList.remove("hidden");
        // Make correct answer required for direct questions
        dialogOverlay
          .querySelector("#correct-answer")
          .setAttribute("required", "true");
      } else if (e.target.value === "mcq") {
        mcqFields.classList.remove("hidden");
        // Remove required from correct answer for MCQ
        dialogOverlay
          .querySelector("#correct-answer")
          .removeAttribute("required");
      }
    });

  // Add option button
  dialogOverlay
    .querySelector("#add-option-btn")
    .addEventListener("click", () => {
      const optionsContainer =
        dialogOverlay.querySelector("#options-container");
      const newOption = document.createElement("div");
      newOption.className = "option-item";
      newOption.innerHTML = `
      <input type="text" class="option-input"
placeholder="Option text">
      <input type="checkbox" class="correct-option">
      <span>Correct</span>
      <button type="button" class="remove-option-
btn">&times;</button>
    `;
      optionsContainer.appendChild(newOption);

      newOption
        .querySelector(".remove-option-btn")
        .addEventListener("click", () => {
          optionsContainer.removeChild(newOption);
        });
```

```
    });

  // Set up remove option buttons for existing options
  dialogOverlay.querySelectorAll(".remove-option-
btn").forEach((btn) => {
    btn.addEventListener("click", () => {
      const optionItem = btn.closest(".option-item");
      const optionsContainer = optionItem.parentElement;
      if (optionsContainer.children.length > 2) {
        optionsContainer.removeChild(optionItem);
      } else {
        showNotification("MCQ questions must have at least 2
options", "error");
      }
    });
  });

  // File attachment preview
  dialogOverlay
    .querySelector("#question-attachment")
    .addEventListener("change", async (e) => {
      const file = e.target.files[0];
      if (!file) return;

      const preview =
dialogOverlay.querySelector("#attachment-preview");
      preview.innerHTML = `<div class="loading-
attachment">Uploading attachment...</div>`;
      preview.classList.remove("hidden");

      try {
        const response = await
uploadQuestionAttachment(file);
        const attachment = response.attachment;

        preview.innerHTML = `
        <div class="attachment-info">
          <span class="attachment-name">${file.name}</span>
```

```
          <span class="attachment-
type">(${attachment.type})</span>
          <button type="button" class="remove-attachment-
btn">&times;</button>
        </div>
        <input type="hidden" id="attachment-data"
value='${JSON.stringify(
          attachment
        )}'>
      `;

        preview
          .querySelector(".remove-attachment-btn")
          .addEventListener("click", () => {
            preview.innerHTML = "";
            preview.classList.add("hidden");
            dialogOverlay.querySelector("#question-
attachment").value = "";
          });
      } catch (error) {
        console.error("Error uploading attachment:", error);
        preview.innerHTML = `
        <div class="attachment-error">
          Failed to upload attachment: ${error.message}
          <button type="button" class="retry-attachment-
btn">Retry</button>
        </div>
      `;

        preview
          .querySelector(".retry-attachment-btn")
          .addEventListener("click", () => {
            dialogOverlay.querySelector("#question-
attachment").click();
          });
      }
    });
```

```
  // Set up remove attachment button if it exists
  const removeAttachmentBtn = dialogOverlay.querySelector(
    ".remove-attachment-btn"
  );
  if (removeAttachmentBtn) {
    removeAttachmentBtn.addEventListener("click", () => {
      const preview =
dialogOverlay.querySelector("#attachment-preview");
      preview.innerHTML = "";
      preview.classList.add("hidden");
      dialogOverlay.querySelector("#question-
attachment").value = "";
    });
  }

  // Form submission
  dialogOverlay
    .querySelector("#question-form")
    .addEventListener("submit", async (e) => {
      e.preventDefault();
      const form = e.target;
      const submitBtn =
form.querySelector('button[type="submit"]');
      submitBtn.disabled = true;
      submitBtn.textContent = "Saving...";

      try {
        // Get the examId from the hidden input field
        const examId = form.querySelector("#exam-id").value;

        if (!examId) {
          throw new Error("Exam ID not found");
        }

        const questionData = getQuestionFormData(form,
examId);
        let result;
```

```
        if (form.querySelector("#question-id").value) {
          // Update existing question
          const questionId = form.querySelector("#question-
id").value;
          result = await updateQuestion(questionId,
questionData);
          showNotification("Question updated successfully",
"success");
        } else {
          // Create new question
          result = await createQuestion(questionData);
          showNotification("Question created successfully",
"success");
        }

        // Close the dialog
        document.body.removeChild(dialogOverlay);

        // Refresh the exam view with the latest questions
        await renderExamEditor(examId);
      } catch (error) {
        console.error("Error saving question:", error);
        showNotification(`Failed to save question:
${error.message}`, "error");
        submitBtn.disabled = false;
        submitBtn.textContent = "Save Question";
      }
    });

  console.log("Question dialog opened successfully");
}

/**
 * Gets the question data from the form
 * @param {HTMLFormElement} form - The form element
 * @param {string} examId - The ID of the exam
 * @returns {Object} - The question data
 */
```

```
function getQuestionFormData(form, examId) {
  const type = form.querySelector("#question-type").value;

  const questionData = {
    examId,
    text: form.querySelector("#question-text").value,
    type,
    points: Number.parseInt(form.querySelector("#question-
points").value),
    timeLimit: Number.parseInt(form.querySelector("#time-
limit").value),
  };

  // Handle attachment
  const attachmentPreview = form.querySelector("#attachment-
preview");
  if (!attachmentPreview.classList.contains("hidden")) {
    const attachmentData = form.querySelector("#attachment-
data");
    if (attachmentData) {
      questionData.attachment =
JSON.parse(attachmentData.value);
    }
  }

  // Handle type-specific fields
  if (type === "direct") {
    questionData.correctAnswer =
form.querySelector("#correct-answer").value;
    questionData.tolerance = Number.parseInt(
      form.querySelector("#tolerance").value
    );
  } else if (type === "mcq") {
    const options = [];
    const correctOptions = [];

    form.querySelectorAll(".option-item").forEach((item,
index) => {
```

```
      const optionText = item.querySelector(".option-
input").value.trim();
      if (optionText) {
        options.push(optionText);
        if (item.querySelector(".correct-option").checked) {
          correctOptions.push(options.length - 1); // Use
the actual index in the final array
        }
      }
    });

    if (options.length < 2) {
      throw new Error("MCQ questions must have at least 2
options");
    }

    if (correctOptions.length < 1) {
      throw new Error("MCQ questions must have at least 1
correct option");
    }

    questionData.options = options;
    questionData.correctOptions = correctOptions;
  }

  return questionData;
}

/**
 * Shows a notification message
 * @param {string} message - The message to display
 * @param {string} type - The type of notification (success,
error, info)
 */
function showNotification(message, type = "info") {
  // Check if notification container exists, create if not
  let notificationContainer =
document.querySelector(".notification-container");
```

```
  if (!notificationContainer) {
    notificationContainer = document.createElement("div");
    notificationContainer.className = "notification-
container";
    document.body.appendChild(notificationContainer);
  }

  // Create notification element
  const notification = document.createElement("div");
  notification.className = `notification ${type}`;
  notification.innerHTML = `
    <div class="notification-content">
      <span class="notification-message">${message}</span>
      <button class="notification-close">&times;</button>
    </div>
  `;

  // Add to container
  notificationContainer.appendChild(notification);

  // Add close button functionality
  notification
    .querySelector(".notification-close")
    .addEventListener("click", () => {
      notification.classList.add("fade-out");
      setTimeout(() => {
        notificationContainer.removeChild(notification);
      }, 300);
    });

  // Auto remove after 5 seconds
  setTimeout(() => {
    if (notification.parentNode) {
      notification.classList.add("fade-out");
      setTimeout(() => {
        if (notification.parentNode) {
          notificationContainer.removeChild(notification);
        }
```

```
      }, 300);
    }
  }, 5000);
}
```

CONCLUSION:

The Examination Management System successfully achieves its objective of providing a digital platform for conducting and managing examinations in an efficient and secure manner. The system simplifies the overall examination process by automating tasks such as exam creation, question management, student participation, answer submission, and result generation.

Through this project, the traditional manual examination system is replaced with a modern, technology-driven solution that saves time, reduces human errors, and ensures accuracy. The system provides a user-friendly interface for both students and teachers, making it easy to use and manage.

The use of modern technologies such as Node.js, Express.js, and MongoDB ensures scalability, flexibility, and efficient data handling. Security features like authentication and OTP-based password recovery enhance the reliability of the system.

Overall, the project demonstrates how digital solutions can improve the quality and efficiency of examination systems. It lays a strong foundation for future enhancements such as online proctoring, mobile application support, and advanced analytics.

BIBLIOGRAPHY:

The following books and online resources were referred to during the analysis, design, and development of the Examination Management System.

Books

- *Web Development with Node and Express*
  by Ethan Brown
- *MongoDB: The Definitive Guide*
  by Shannon Bradshaw, Eoin Brazil, and Kristina Chodorow
- *JavaScript: The Good Parts*
  by Douglas Crockford
- *Eloquent JavaScript*
  by Marijn Haverbeke

Websites

- https://www.google.com
- https://www.w3schools.com
- https://developer.mozilla.org
- https://www.stackoverflow.com
- https://expressjs.com
- https://www.mongodb.com